



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique

Université des Sciences et de la Technologie Houari
Boumediène
Faculté d'Informatique
Département d'Intelligence Artificielle et Sciences des
Données (DIASD)



Master 2 Systèmes Informatiques Intelligents

Data Mining Project

**Forest Fire Prediction using Soil, Climate, Elevation
and Land Cover Data**

Présenté par :

Anane Dalia Khadidja
Moussaoui Sarah

Groupe 1

Responsable du TP :

N.Houacine

Année universitaire 2025–2026

1	Data Extraction	1
1.1	Overview	1
1.2	Soil Data Extraction	1
1.3	Climate Data	1
1.4	Elevation Data Extraction	2
1.4.1	Data Preparation and Raster Clipping Workflow	2
1.5	Land Cover Data Extraction	2
1.5.1	Land Cover Data Standardization and Integration	2
1.6	Fire Data Extraction	3
2	Data Description And Visualisation	4
2.1	Overview	4
2.2	Soil data description and visualisation	4
2.3	Soil Dataset Description	4
2.4	Soil visualisation	5
2.4.1	Histogram-based distribution analysis	6
2.5	Climate data description and visualisation	8
2.5.1	Data Description	8
2.5.2	Data Visualization	8
2.6	Elevation data description and visualisation	9
2.6.1	Data Loading and Integrity Check	9
2.6.2	Descriptive Statistics and Visualisation	10
2.7	Land cover data description and visualisation	10
2.7.1	Data Structure and Integrity	10
2.7.2	Initial Visualization and Area Analysis	11
2.8	Fire data description and visualisation	12
2.8.1	Data Loading and Inspection	12
2.8.2	Exploratory Data Analysis (EDA) and Visualization	12
2.9	Summary	12
3	Data Cleaning and Initial Feature Preparation	13
3.1	Overview	13
3.2	Cleaning The Soil Dataset	13
3.2.1	Handling missing values	13
3.2.2	Handling Outliers and Invalid Values	15
3.2.3	Initiation to Feature Engineering: Identification of Redundant Variables	19
3.3	Climate Data Cleaning and Missing Value Treatment	22

3.3.1	Identification of Missing Values	22
3.3.2	Imputation Strategy	22
3.4	Land Cover Data Cleaning and Feature Preparation	22
3.4.1	Classification Code Harmonization	22
3.5	Fire Feature Preparation	23
4	Data Integration and Preprocessing	24
4.1	Overview	24
4.2	Data Merging Procedure	24
4.3	Splitting and Resampling techniques	25
4.3.1	Splitting to training test and validation set	25
4.4	Preprocessing and Normalization	26
4.4.1	Soil Properties:	26
4.4.2	Visualization of Normalization Effects	26
4.4.3	Climate Features:	27
4.4.4	Expansion of Monthly Climate Variables	27
4.4.5	Elevation Features:	28
4.4.6	Saving the Normalized Data	28
5	Feature Engineering	29
5.1	Overview	29
5.2	One-Hot Encoding of Land Cover (High-Level Classes)	29
5.3	Dropping Unnecessary Columns	29
6	Supervised Machine Learning	30
6.1	Overview	30
6.2	Decision Tree Results	30
6.2.1	Scratch Implementation Results	30
6.2.2	Scikit-Learn Implementation	31
6.3	KNN Model Configuration and Evaluation	31
6.3.1	Model Performance	31
6.4	Random Forest	32
6.4.1	Random Forest – From Scratch Implementation	32
6.4.2	Scikit-learn Implementation	32
7	Unsupervised Machine Learning	33
7.1	Overview	33
7.2	CLARA K-Medoids Clustering	33
7.2.1	CLARA Scikit-learn Implementation and Results	33
7.2.2	Bayesian Hyperparameter Optimization	34
7.2.3	From-Scratch Implementation of CLARANS	34
7.3	DBSCAN Model Configuration and Evaluation	35
7.3.1	DBSCAN Performance Metrics	36
7.4	K-Means Model Configuration and Evaluation	36
7.4.1	K-Means Performance Metrics	37
A	General Annexes	
A.1	Soil Data Extraction from HWSD2	
A.1.1	Implementation environment	
A.1.2	Input HWSD2 data and file structure	
A.1.3	Mapping units (MU / SMU) as join keys	
A.1.4	Construction of a 0.1° mapping-unit grid	

A.1.5	Extraction of topsoil (D1) properties from the HWSD2 database
A.1.6	Join with the 0.1° grid and export
A.1.7	Summary
A.2	Normality assessment using quantiles plots
A.3	Elevation dataset description and Visualisation
A.3.1	Distribution and Topographical Profile
A.4	Fire dara description and visualization
A.4.1	Variable Definitions and Geospatial Metrics
A.4.2	Exploratory Data Analysis
A.5	Climate Data Visualization Summary
A.6	Soil Data Cleaning Summary
A.6.1	Spatial Visualization of Soil Variables
A.6.2	BallTree Algorithm in the Context of Soil Data
A.6.3	Residual Outliers After Imputation
A.7	Resampling Methods
A.8	K-Nearest Neighbors (KNN) Model Visualizations
A.9	Decision Tree
A.9.1	From scratch Implementation
A.10	Random forest
A.10.1	From-Scratch Implementation
A.10.2	Scikit-learn Implementation
A.10.3	Bayesian Optimization for Random Forest
A.10.4	Discussion
A.11	Visualisations of K-Means results
A.11.1	Bayesian Optimization and Optimized K-Means ($K = 8$)
A.11.2	From-Scratch K-Means Clustering ($K = 5$)
A.11.3	Scikit-Learn K-Means Clustering ($K = 3$)
A.11.4	Overall Clustering Assessment
A.12	DBSCAN Clustering Visualizations
A.12.1	DBSCAN with Alternative Parameterization
A.12.2	DBSCAN vs. Ground Truth (PCA Projection, Scikit-Learn) and (PCA Projection, From-Scratch)
A.12.3	DBSCAN Detailed Analysis (Optimized Configuration)
A.12.4	Bayesian Optimization and Optimized DBSCAN
A.12.5	Overall DBSCAN Conclusion
A.13	Clara K-Medoids Clustering
A.13.1	Scikit-learn Implementation
A.13.2	Bayesian Optimization
A.13.3	From-Scratch Implementation of CLARANS

1.1 Overview

In this chapter, we describe the extraction of all datasets used in the project: soil, climate, elevation, land cover, administrative boundaries, and historical fire events.

We detail the tools, sources, coordinate reference systems, and the spatial and temporal extents chosen for each layer.

1.2 Soil Data Extraction

The soil dataset was extracted from HWSD2 by resampling the categorical soil raster to a consistent 0.1° grid over the Algeria–Tunisia AOI using mode-based resampling. GeoPandas, shapely, rioxarray, and rasterio were used for AOI handling, raster clipping, reprojection, and grid construction. The HWSD2 Microsoft Access database was queried via pyodbc to retrieve topsoil (D1, 0–20 cm) physical and chemical properties linked through soil mapping unit IDs. NumPy and pandas handled data cleaning, type normalization, joins, and tabular exports, while matplotlib supported rapid spatial sanity checks. The final output is a GIS- and ML-ready soil feature dataset exported as CSV and GeoPackage for downstream fire prediction models. The detailed extraction method Implementation is provided in Annex A.1.

1.3 Climate Data

Monthly climate variables were extracted from the *WorldClim v2.1* global dataset, using the monthly weather product available at a spatial resolution of 5 arc-minutes (approximately 10 km). The variables considered include total precipitation, minimum temperature, and maximum temperature, provided as monthly GeoTIFF rasters.

An Area of Interest (AOI) covering Algeria and Tunisia was defined from a GeoPackage and all spatial layers were standardized to the WGS84 geographic coordinate system (EPSG:4326) to ensure spatial consistency. A regular spatial grid was then constructed over the AOI, with a fixed angular resolution, and intersected with the AOI boundary. Each grid cell was assigned a unique identifier (`cell_id`), and the centroid of each cell was computed to serve as a representative sampling location.

For each monthly raster file, the data were processed through a standardized pipeline: the raster was opened using `rioxarray`, assigned a coordinate reference system if missing, reprojected to EPSG:4326 when necessary using bilinear resampling, and clipped to the AOI geometry. Climate values were then extracted at the grid centroids using nearest-neighbor sampling,

yielding one observation per grid cell and per month.

Temporal information was derived directly from raster filenames, with the year and month parsed and stored explicitly in the output tables. The resulting datasets were organized in a tidy long format (`cell_id`, `year`, `month`, `value`) and exported primarily in Parquet format, with an automatic fallback to CSV when required. Basic validation checks were performed throughout the process, including verification of value ranges before and after clipping, assessment of missing data ratios, and inspection of sampled values for selected months to ensure consistency.

1.4 Elevation Data Extraction

The analysis utilizes a **Digital Elevation Model (DEM)** derived from an **ESRI GRID (ArcInfo GRID)** dataset. This format is a widely accepted raster structure for representing surface elevation. The primary objective of this processing stage was to efficiently extract, or **clip**, the relevant elevation data corresponding precisely to the merged geopolitical boundaries of Algeria and Tunisia.

1.4.1 Data Preparation and Raster Clipping Workflow

The preprocessing workflow began with the configuration of environment variables to manage dataset paths and ensure portability across processing stages. The geographical area of interest (AOI) was defined by loading merged country boundaries from a GeoPackage file (`aoi_dza_tun.gpkg`) into a `GeoDataFrame` using `geopandas`. In parallel, the source elevation raster was loaded from the `be15_grd` directory using `rasterio`, allowing access to its spatial metadata, including the native Coordinate Reference System (CRS).

To enable accurate spatial operations, the vector boundaries were reprojected to match the raster's CRS. This alignment step is essential, as raster masking requires both datasets to share the same spatial reference. Once the coordinate systems were unified, the elevation raster was clipped using the AOI geometries through the `rasterio.mask.mask()` function, producing a masked elevation array and an updated spatial transformation.

Finally, the original raster metadata was updated to reflect the new dimensions and georeferencing parameters of the clipped dataset. The output format was explicitly set to `GeoTIFF` to ensure write compatibility and standardization. The resulting digital elevation model was saved as `elevation_dz_tn.tif`, constituting the final processed dataset ready for subsequent topographical or hydrological analyses.

1.5 Land Cover Data Extraction

This section details the workflow for processing and integrating the Land Cover (LC) data for the study area. Unlike the Elevation dataset (a raster), the Land Cover data was acquired in a **vector format** (ESRI Shapefiles), necessitating specific preparation steps to ensure geometric and attribute consistency for subsequent analysis.

1.5.1 Land Cover Data Standardization and Integration

Land Cover data for Algeria and Tunisia were initially provided as separate ESRI Shapefiles. To improve processing efficiency and ensure consistency, both datasets were standardized to the **GeoJSON** format prior to integration. The Algerian Shapefile (`dza_gc_adg.shp`) was directly imported into a `geopandas GeoDataFrame` and exported as `landcover_algeria.geojson`. For Tunisia, the Shapefile (`tun_gc_adg.shp`) was first optionally simplified using the `geometry.simplify(0.001, preserve_topology=True)` method to reduce polygon complexity and file size while preserving topological integrity, before being saved as `landcover_tunisia.geojson`.

Following standardization, the two GeoJSON datasets were merged into a unified Land Cover layer. Prior to concatenation, a harmonization process was applied to ensure structural and spatial compatibility. This included converting all attribute names to lowercase to avoid schema mismatches, and renaming semantically equivalent fields to a common standard. The harmonized GeoDataFrames were then concatenated using `pd.concat()` to form the complete Land Cover dataset. Finally, an optional `dissolve()` operation was applied to generate a single continuous boundary, facilitating regional extent visualization and spatial analysis.

1.6 Fire Data Extraction

The dataset utilized for this analysis is a composite structure derived from two distinct data sources corresponding to fire incidents recorded in **Algeria** and **Tunisia**. The original data for each country is provided in a separate CSV file.

Data Integration and Structure

The core process involved the simple integration of these two national datasets to form a unified, comprehensive file. This merged dataset provides a consistent structure across all observations, which is essential for comparative analysis.

The resultant dataset comprises 15 columns, detailing geographical, temporal, and radiative properties of the detected fire events.

The complete set of variables is as follows:

- `latitude, longitude`: Geographical coordinates of the fire pixel.
- `acq_date, acq_time`: Time and date of acquisition.
- `satellite, instrument`: Sensor and platform used for detection (e.g., N20, VIIRS).
- `bright_ti4, bright_ti5`: Brightness temperatures (in Kelvin) measured by channels I4 and I5.
- `frp`: Fire Radiative Power (in MW), a key metric for fire intensity.
- `scan, track`: Dimensions of the fire pixel (in km).
- `confidence`: Confidence level of the fire detection.
- `version, daynight, type`: Auxiliary identifiers.

CHAPTER 2

DATA DESCRIPTION AND VISUALISATION

2.1 Overview

This chapter presents the datasets used in the fire prediction task (soil, climate, elevation, land cover and fire occurrence) and the main visual explorations carried out on them. For each data source we describe:

- the original format and spatial/temporal coverage,
- the extraction and preprocessing steps applied to build the common 0.1° grid over Algeria-Tunisia,
- a set of exploratory visualisations highlighting key spatial patterns and potential relationships with fire activity.

The goal is to provide a clear picture of what information is available in each layer and how it behaves over the study area before moving to the modelling stage.

2.2 Soil data description and visualisation

2.3 Soil Dataset Description

The soil dataset used in this study consists of **23,262 grid cells** (0.1° resolution) covering Algeria and Tunisia. Each row corresponds to a regular grid-cell centroid (x, y) together with topsoil (0–20 cm) physical and chemical attributes extracted from the Harmonized World Soil Database v2 (HWSD2). The total memory usage for the soil data is approximately **5.90 MB**. All features originate either from the D1 layer (topsoil) or categorical summaries at the Soil Mapping Unit (SMU) level.

Spatial Columns

x, y (*longitude, latitude*) specify the centroid of each cell. These coordinates enable spatial alignment with land cover, climate and ignition datasets used in fire modelling [Giglio et al., 2013].

Identification Key

HWSD2_SMU_ID uniquely identifies the soil mapping unit associated with each cell. This ID links the grid to HWSD2 pedological information and ensures consistent reproducibility [FAO and IIASA, 2021].

Texture and Particle-Size Distribution

COARSE (%) represents the proportion of fragments > 2 mm, influencing water retention and drying rates. High coarse content increases surface dryness and ignition probability [Prieto-Fernandez, 2019]. **SAND**, **SILT** and **CLAY** (%) describe the granulometric composition. Sandy soils drain rapidly and dry faster, whereas clay-rich soils retain moisture and generally reduce fire susceptibility [Bowman, 2009, Saatchi, 2012]. **TEXTURE_USDA** and **TEXTURE_SOTER** provide categorical texture classes (USDA and SOTER standards). Texture strongly regulates infiltration, soil moisture availability and vegetation composition [NRCS, 2017, Batjes, 2016].

Density and Structural Properties

BULK (g cm^{-3}) measures dry soil density. Higher bulk density limits root penetration and reduces water storage capacity [Certini, 2005a]. **REF_BULK** is the expected bulk density estimated from pedotransfer functions, used to detect compaction anomalies [Reynolds, 2018].

Organic Matter and Nutrients

ORG_CARBON (%) represents organic carbon content, a driver of both soil moisture retention and surface fuel accumulation [De Groot, 2009]. **TOTAL_N** (%) influences vegetation productivity and post-fire biomass recovery [Pausas and Bradstock, 2012]. **CN_RATIO** (C:N) governs decomposition rates; high ratios slow down decay and preserve surface litter, increasing flammability [Fernandez and Vega, 2018].

Chemical and Exchange Properties

PH_WATER indicates soil acidity. Extreme pH affects nutrient cycling and decomposition, altering fuel loads [Certini, 2005b]. **CEC_SOIL**, **CEC_CLAY** and **CEC_EFF** (cmol kg^{-1}) represent nutrient-holding capacity. High CEC improves water retention and thus reduces fire proneness [Brady and Weil, 2017, Jenny, 1941]. **TEB** (Total Exchangeable Bases) and **BSAT** (%) describe soil fertility; productive soils often accumulate more biomass, increasing available fuel [Archibald, 2013]. **ALUM_SAT** (%) and **ESP** (%) capture aluminum and sodium saturation. High ESP causes soil crusting and sparse vegetation, producing patchy fire propagation patterns [Qadir, 2014]. **TCARBON_EQ** (total carbon equivalents) and **GYPSUM** (%) indicate soil development stage in arid systems. Gypsiferous and carbonate-rich soils often host low vegetation cover and reduced fire intensity [Herrero, 2009]. **ELEC_COND** (dS m^{-1}) measures salinity; high salinity stresses vegetation and correlates with lower fuel loads [Ruiz-Sinoga, 2012].

Overall, these variables collectively describe the soil moisture regime, nutrient status, vegetation-support capacity and physical structure of the topsoil, all of which are key environmental determinants of wildfire ignition and spread.

2.4 Soil visualisation

This section presents an exploratory visual analysis of the soil variables. Two complementary approaches are used: distribution analysis through histograms and normality assessment through quantiles plots. All analyses are based on the 0.1° grid covering the study area.

2.4.1 Histogram-based distribution analysis

Figure 2.1 shows the histograms of all numeric soil features together with their mean, median and mode.

The **COARSE** fraction exhibits a slightly right-skewed distribution (mean > median > mode), indicating that most soils contain low proportions of coarse fragments, with a limited number of rocky soils forming the upper tail. **SAND** follows a right-skewed distribution, with a dominance of moderately sandy soils and a tail toward very sandy conditions. **SILT** displays a slightly left-skewed shape (mode > median > mean), suggesting a concentration around medium silt contents with few low-silt extremes. **CLAY** is right-skewed, showing a prevalence of fine to medium clay contents and a small proportion of highly clay-rich soils.

BULK density presents a right-skewed distribution, with most values clustered around typical mineral soil densities and a tail toward compacted soils. **REF_BULK** shows a very similar pattern but with weaker skewness, indicating a more homogeneous reference density. **ORG_CARBON** is clearly right-skewed, reflecting the dominance of carbon-poor soils and a long tail corresponding to organic-rich zones. **PH_WATER** is left-skewed, with most soils near neutral to alkaline conditions and a smaller acidic tail.

TOTAL_N follows a right-skewed distribution, indicating nitrogen-poor conditions over most of the area, with a few fertile hotspots. **CN_RATIO** shows slight right skewness, with most soils presenting balanced ratios and a limited number of high C:N values. **CEC_SOIL** is right-skewed, dominated by moderate cation exchange capacities and a long tail toward high-CEC soils. **CEC_CLAY** is slightly left-skewed, indicating generally high clay activity with fewer low-activity mineral clays.

CEC_EFF and **TEB** both show right-skewed distributions, with a small number of soils exhibiting very high effective exchange capacity or base content. **BSAT** presents a bimodal to slightly right-skewed distribution, with a major peak near full base saturation and a secondary lower-saturation group, reflecting widespread alkaline conditions. **ALUM_SAT** is strongly right-skewed, with very low aluminum saturation in most soils and only a few acidic outliers. **ESP** also exhibits right skewness, indicating generally low sodicity with isolated saline or sodic areas. **TCARBON_EQ** shows slight right skewness, corresponding to moderately carbonated soils with a few carbonate-rich extremes. **GYP SUM** and **ELEC_COND** are strongly right-skewed, highlighting that most soils are gypsum-free and non-saline, while a small number of cells represent gypsiferous or saline environments.

Overall, only a limited subset of variables (**REF_BULK**, **PH_WATER**, **SILT**, **CN_RATIO**, **CEC_CLAY**) approach a symmetric or near-normal distribution, while the majority exhibit clear positive skewness.

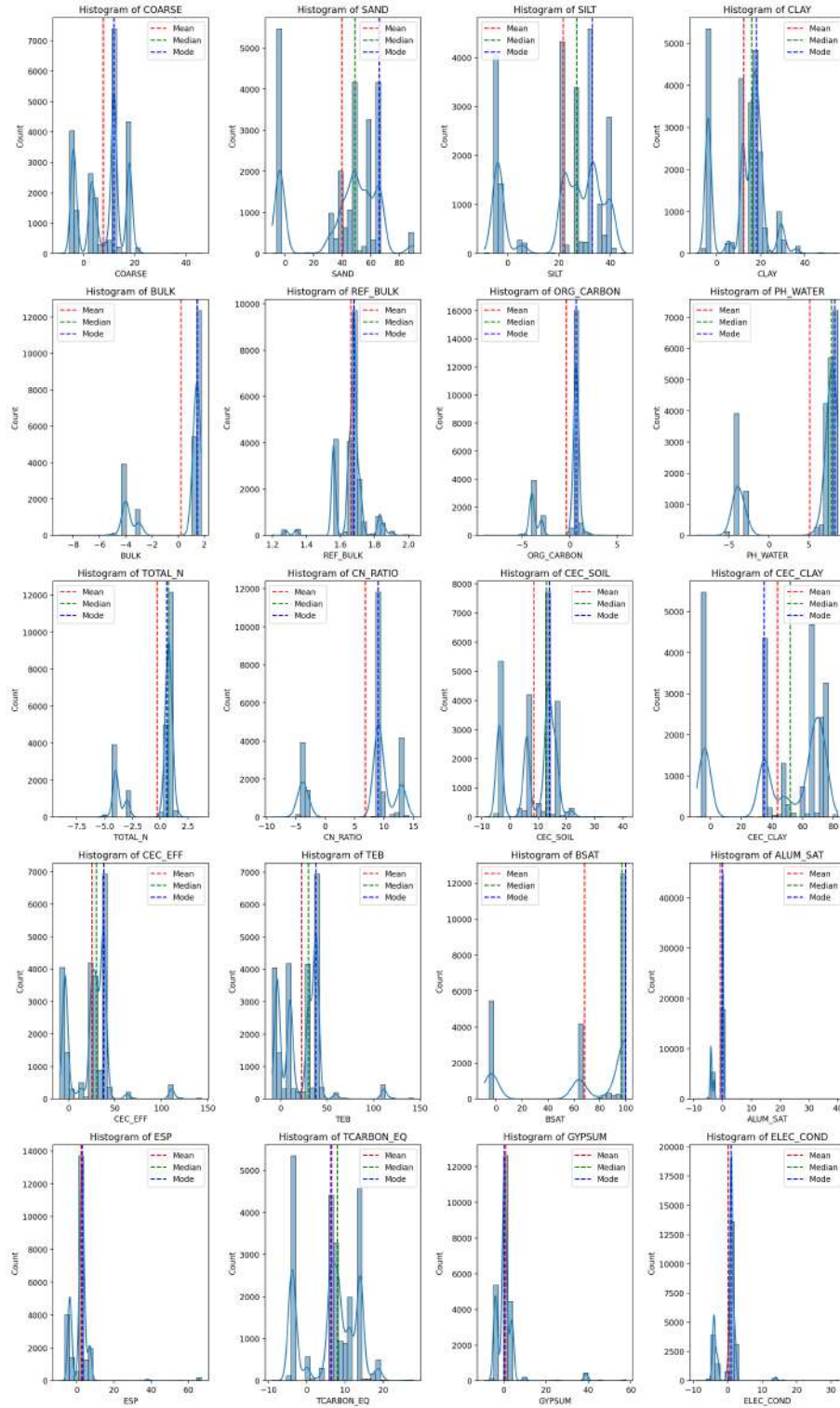


Figure 2.1: Histograms of soil numeric variables with mean, median and mode indicators.

Quantiles plots provide a complementary perspective on normality, as shown in Annex A.2.

2.5 Climate data description and visualisation

2.5.1 Data Description

Climate variables were extracted from the *WorldClim v2.1* monthly weather dataset at a spatial resolution of 5 arc-minutes (approximately 10 km). The study area covers Algeria and Tunisia, and the temporal scope spans the period 2020–2024. Three monthly climate variables were considered:

- Total precipitation (`prec_mm`, in millimeters),
- Maximum temperature (`tmax_c`, in degrees Celsius),
- Minimum temperature (`tmin_c`, in degrees Celsius).

Each variable was sampled on a common spatial grid using centroid-based extraction, yielding identical spatial coverage and temporal resolution. The resulting datasets have identical shapes:

- Precipitation: (1 399 320, 4),
- Maximum temperature: (1 399 320, 4),
- Minimum temperature: (1 399 320, 4).

Each dataset occupies approximately 42.7 MB in memory. All temperature values are expressed in degrees Celsius, ensuring physical interpretability and consistency across analyses.

2.5.2 Data Visualization

Distribution Analysis

Figure 2.2 presents the empirical distributions of precipitation, maximum temperature, and minimum temperature. Precipitation exhibits a highly right-skewed distribution, with a large concentration of low values and a long tail corresponding to intense rainfall events. This behavior is typical of semi-arid and Mediterranean climates, where dry conditions dominate most months.

Maximum and minimum temperatures display broader, multi-modal distributions reflecting strong seasonal variability. The separation between cold and warm regimes is clearly visible, with minimum temperatures spanning from sub-zero values to above 30°C, and maximum temperatures reaching nearly 50°C in extreme summer conditions.

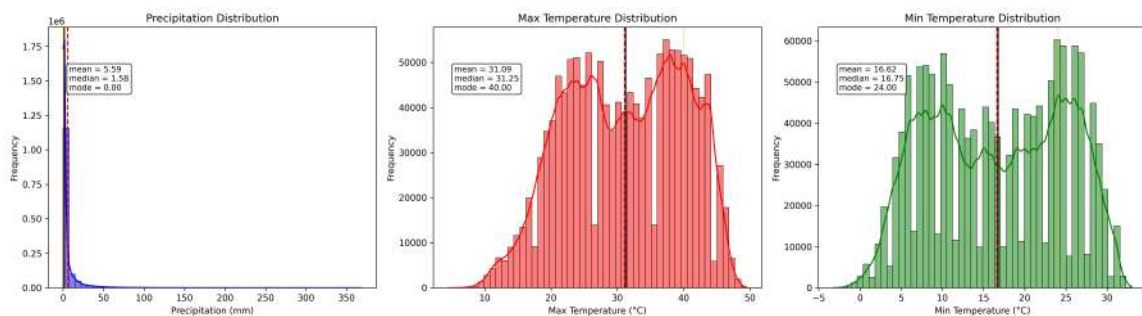


Figure 2.2: Histograms of raw climate variables: precipitation, maximum temperature, and minimum temperature.

Kernel Density Estimation (KDE) plots after imputation (Figure 2.3) confirm these observations. Precipitation remains strongly right-skewed, while temperature variables show smoother, multi-peaked density functions corresponding to seasonal cycles.

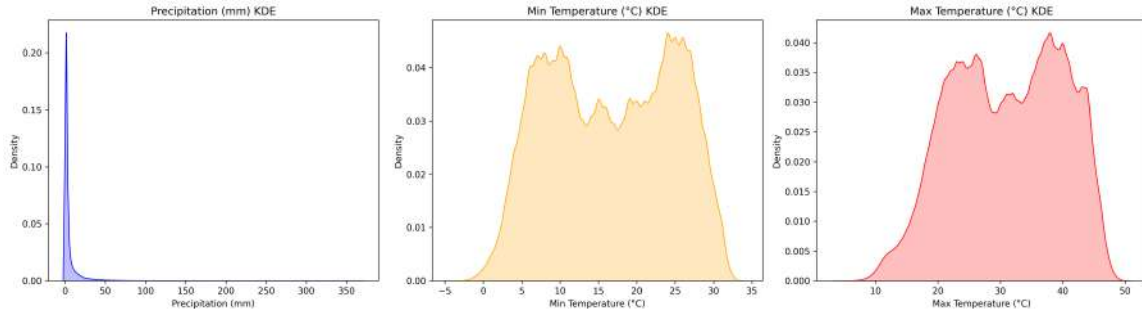


Figure 2.3: Kernel density estimates of imputed climate variables.

Normality Assessment

Q-Q plots (Figure 2.4) indicate strong deviations from normality for precipitation, particularly in the upper quantiles, confirming the presence of heavy tails. Temperature variables exhibit more linear behavior in central quantiles but still deviate at the extremes, reflecting bounded physical processes and seasonal effects. These results indicate that no climate variable follows a Gaussian distribution, justifying the use of normalization or robust scaling rather than assumptions of normality.

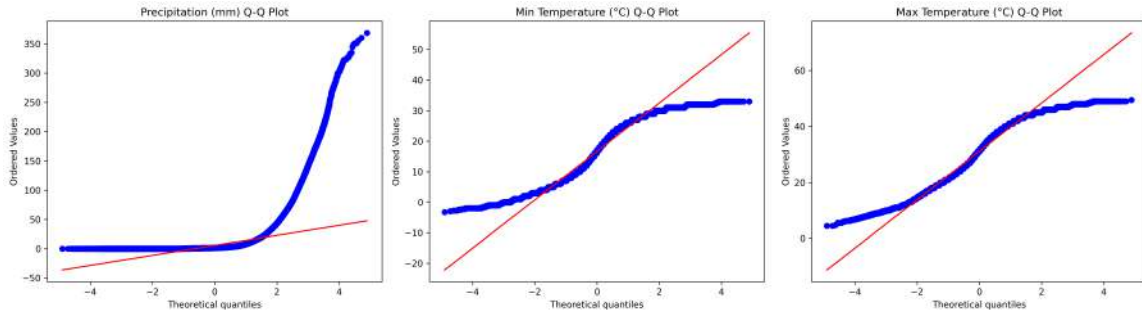


Figure 2.4: Q-Q plots for precipitation, minimum temperature, and maximum temperature after imputation.

Spatial Patterns

The figure in the Annex A.5 illustrates spatial heatmaps of mean climate conditions over the 2020–2024 period. Precipitation shows a strong north–south gradient, with wetter conditions in northern coastal and mountainous regions and extremely dry conditions in the Saharan south. Temperature maps display the inverse pattern: higher maximum and minimum temperatures dominate southern regions, while cooler conditions prevail at higher latitudes and elevations.

Monthly snapshot maps further highlight seasonal contrasts, particularly for January, where temperature gradients are most pronounced.

2.6 Elevation data description and visualisation

This section details the analysis and statistical exploration of the extracted **Elevation Dataset** for the combined region of Algeria and Tunisia. The dataset is examined for data integrity, statistical properties, and spatial distribution.

2.6.1 Data Loading and Integrity Check

The analysis begins by loading the processed raster file using `rasterio`.

- **No-Data Handling:** An integrity check was performed to identify and quantify the raster’s no-data values (`nodata_value`). These values, often representing areas outside the clipped boundary or sea level, were masked out during statistical calculations to ensure accurate topographical analysis.

2.6.2 Descriptive Statistics and Visualisation

Core statistics were computed on the valid elevation pixels to characterize the overall terrain:

- Mean Elevation: 538.327
- Median Elevation: 464.0
- Mode Elevation: 301

The **Elevation Dataset** was visualized to create a professional elevation map. The map utilizes a color scale to intuitively display elevation in meters.

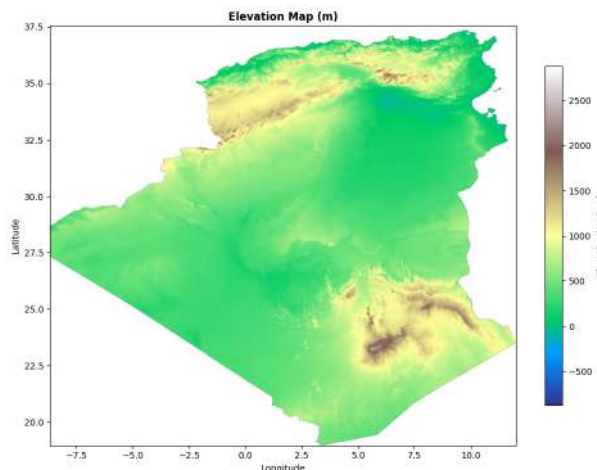


Figure 2.5: Color-ramped visualization of the Elevation Dataset for the combined Algeria and Tunisia study area. The `terrain` color map illustrates the topographical gradient, ranging from low coastal plains to high inland mountains.

2.7 Land cover data description and visualisation

2.7.1 Data Structure and Integrity

The merged Land Cover GeoDataFrame (`landcover_gdf`) was loaded and inspected for structural consistency.

- **Data Counts:** The total number of unique features and the distinct values for the primary classification columns—`lcccode` (Land Cover Classification Code) and `gridcode`—were tallied.
- **Redundancy Check:** An inspection for rows with duplicated `id` values was performed. Although duplicates were found, the difference in associated geometry or attributes suggested they were not true feature duplicates, and the `id` column was deemed non-essential for the model and slated for removal.
- **Missing Values:** A check across all columns confirmed the absence of any null/NAN values, indicating completeness.

Relationship between Classification Codes A key analysis focused on the relationship between `gridcode` and `lcccode`. By checking unique pairs of these two columns, it was determined that they possess a one-to-one relationship. Consequently, `gridcode` was identified as a redundant column that provides no additional information and is recommended for dropping during preprocessing.

2.7.2 Initial Visualization and Area Analysis

The distribution of land cover types was visualized to understand the regional composition:

- **Distribution Map:** The Land Cover Map was plotted, coloring polygons based on their `lcccode` to visualize the spatial arrangement of classes.

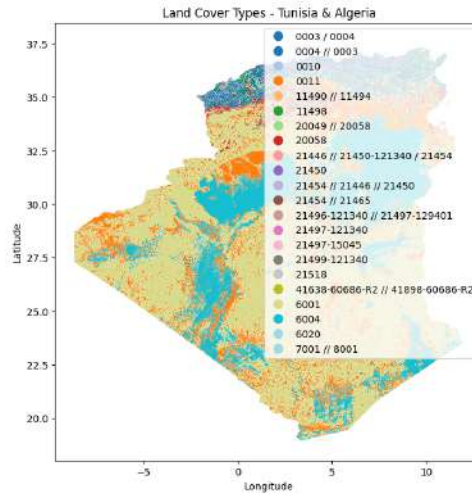


Figure 2.6: Geospatial visualization of the merged Land Cover Map, colored by the specific Land Cover Classification Code (`lcccode`).

- **Area Analysis:** The geometric area of each polygon was calculated. A bar plot was generated showing the Total Area by Land Cover Code.

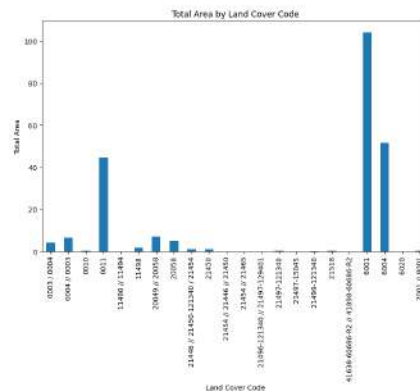


Figure 2.7: Total aggregated area (in square degrees) covered by each Land Cover Code in the combined dataset.

In the land cover plot, the classes coded as **6004**, **6001**, and **0011**, which correspond to **bare land**, occupy the largest proportion of the study area.

2.8 Fire data description and visualisation

2.8.1 Data Loading and Inspection

The analysis began by loading the datasets for Algeria (`df_fire_dza`) and Tunisia (`df_fire_tun`). An initial exploratory inspection was performed using descriptive statistics to assess the distribution and range of key numerical variables, including fire radiative power (`frp`) and brightness temperatures. Data quality was verified by examining data types, memory usage, and the proportion of missing values, confirming structural consistency and integrity across both datasets. Finally, the two datasets were merged using `pd.concat()` into a single `GeoDataFrame` (`df_fire`), enabling unified regional analysis.

2.8.2 Exploratory Data Analysis (EDA) and Visualization

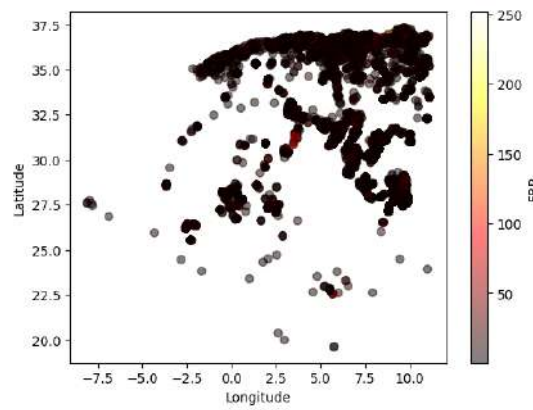


Figure 2.8: Geospatial distribution of fire events across Algeria and Tunisia. Points are colored by Fire Radiative Power (FRP), highlighting areas of highest fire intensity.

The geospatial distribution of fire events was visualized using a scatter plot of `longitude` vs. `latitude`, with the color intensity mapped to `frp`. This visualization is essential for identifying fire clusters and high-intensity regions across the combined study area. As shown in the map, the majority of fire points are concentrated in the northern regions, reflecting higher vegetation density and more favorable climatic conditions for fire occurrence compared to the arid southern areas.

2.9 Summary

Across all datasets, we obtain a harmonised 0.1° grid where each cell is described by soil, climate, topography, land cover and fire occurrence variables. The exploratory maps and distributions presented in this chapter reveal the main gradients (e.g. climatic aridity, vegetation cover, soil texture) that will later be exploited as predictors in the supervised and unsupervised learning experiments.

CHAPTER 3

DATA CLEANING AND INITIAL FEATURE PREPARATION

3.1 Overview

This chapter outlines the data cleaning procedures applied to the raw datasets, including soil, climate, elevation, land cover, and fire data. Each section details the specific cleaning steps taken to prepare the data for analysis, ensuring quality and consistency across all variables.

3.2 Cleaning The Soil Dataset

3.2.1 Handling missing values

Before any modelling step, missing values were inspected and handled in a controlled way. The soil table contains both *numeric* variables (e.g., `SAND`, `CLAY`, `REF_BULK`) and *nominal* (categorical) variables (e.g., `TEXTURE_USDA`, `TEXTURE_SOTER`), which require different strategies.

Global inspection and spatial check

A global missingness score was computed by counting all null cells relative to the total number of cells in the dataset (*missing percentage*). Rows containing at least one missing value were extracted into a dedicated subset to quantify, for each column, the proportion of missing observations. This step allows the identification of which soil variables are responsible for most incomplete rows.

To ensure that missingness was not spatially structured (for example concentrated in a specific region), a compact geospatial visualization was produced. Instead of generating one map per feature, all numeric variables were displayed using a single grid of subplots, providing a global view of their spatial variation while keeping the report concise. see Annex A.6 illustrates these spatial patterns and confirms the expected geographic heterogeneity of soil properties across the study area.

Cross-check of missing patterns for key variables

A targeted diagnostic was performed on two critical attributes: `TEXTURE_USDA` (categorical soil texture class) and `REF_BULK` (reference bulk density). The goal was to verify whether missing values appear jointly or independently. The indices of rows with missing soil data were stored, then highlighted in scatter plots as red points aligned at a baseline. This makes missingness immediately visible as discrete “gaps” in the variable signal across the dataset index. Figure 3.1

shows that missing values are present but limited, and that the variables do not necessarily become missing in the same rows.

A complementary boxplot analysis was used to summarize the distribution of `TEXTURE_USDA` and `REF_BULK` and to identify potential outliers. Histograms were also generated for `TEXTURE_USDA` (before and after imputation) to verify that the overall class structure is preserved. Finally, an ECDF (empirical cumulative distribution function) and a density plot were used for `REF_BULK` to better understand its distribution and quartile structure before imputation.

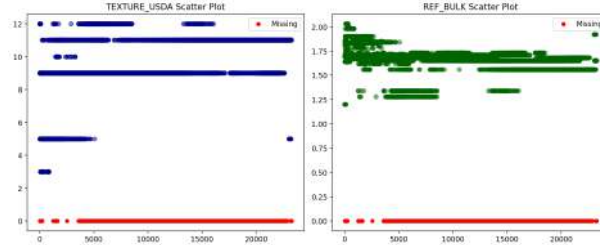


Figure 3.1: Scatter plots showing missing values (red markers) for `TEXTURE_USDA` and `REF_BULK`.

Imputation of `REF_BULK` (numeric)

For `REF_BULK`, missing values were imputed using a simple and robust strategy: **median imputation**. The median is resistant to skewness and extreme values, which is appropriate given the non-Gaussian behaviour observed in several soil variables. To ensure that the imputation did not distort the variable distribution, kernel density estimates were compared before and after the imputer. Figures 3.2 show that the global density shape remains consistent after filling missing values.

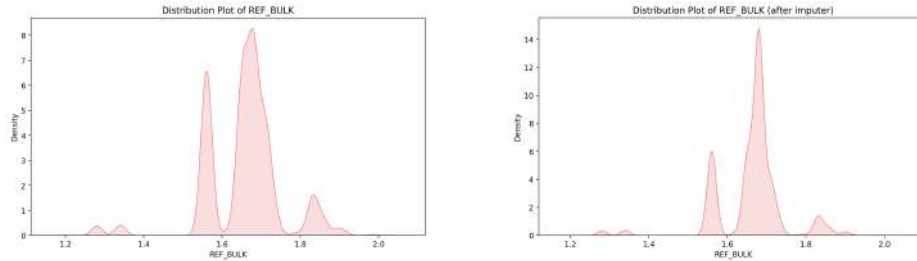


Figure 3.2: Density plot of `REF_BULK` before (left) and after (right) median imputation.

Imputation of `TEXTURE_USDA` (categorical) using composition similarity

For `TEXTURE_USDA`, a direct statistical imputation (mean/median) is not meaningful because the variable is categorical. Instead, a **data-driven imputation** was adopted using the numeric texture composition: `SAND`, `SILT`, and `CLAY`.

Only rows with known `TEXTURE_USDA` were used to learn typical texture “profiles” per class. First, the three composition variables were standardized so that Euclidean distances are computed fairly across dimensions. Then, class centroids were computed as the mean standardized composition for each known USDA class. For each row with missing `TEXTURE_USDA`, its standardized composition was compared to all class centroids, and the closest centroid (minimum Euclidean distance) defined the imputed class. This approach preserves the physical meaning of USDA texture classes, since the imputation is guided by the sand-silt-clay balance rather than arbitrary label filling.

Finally, the remaining number of missing values in `TEXTURE_USDA` was checked to ensure completeness. The class distribution was also recomputed after imputation, and a histogram was

generated to confirm that the imputation did not introduce unrealistic class artefacts. Figures 3.3 show the distribution before and after imputation.

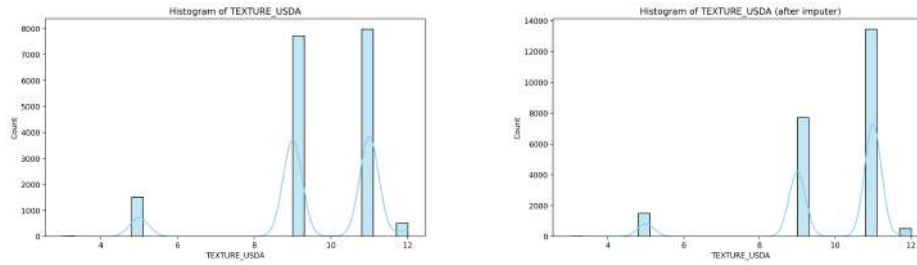


Figure 3.3: Histogram of `TEXTURE_USDA` before imputation (left, known classes only) and after smart imputation using nearest centroid (right, based on `SAND`, `SILT`, `CLAY`).

Rationale

This missing-value strategy is intentionally conservative: median filling for a numeric variable (`REF_BULK`) to minimize distribution drift, and centroid-based imputation for a categorical variable (`TEXTURE_USDA`) to preserve soil-texture semantics. The result is a complete soil feature table that remains consistent with the original spatial and statistical structure of the dataset.

3.2.2 Handling Outliers and Invalid Values

Problem: Negative Values in Soil Properties

During the exploratory analysis of the soil dataset, a large number of outlier values were identified that are physically impossible. In particular, several soil properties exhibited negative values, including bulk density, soil pH, organic carbon content, electrical conductivity, total nitrogen, C:N ratio, base saturation, aluminum saturation, exchangeable sodium percentage, and gypsum content. Such values are invalid from a pedological perspective, as these variables are strictly non-negative by definition (e.g., pH is bounded within a 0–14 scale, and bulk density must be positive).

A systematic inspection of all numeric soil attributes revealed that exactly **5,480 records** contained negative values simultaneously across all key soil chemistry variables. This strong co-occurrence strongly suggests that these records correspond to missing data encoded as negative placeholders or to data ingestion errors rather than true soil measurements. Consequently, these records were flagged as invalid outliers requiring correction.

Detection and Visualization of Invalid Records

Initial outlier detection using a generic statistical function (`detect_outliers`) reported abnormally high counts of extreme values in several columns, such as approximately 10,961 outliers in gypsum and 9,976 in the C:N ratio. These unexpectedly high numbers suggested a structural issue in the dataset rather than isolated anomalies.

To confirm this hypothesis, each numeric soil variable was explicitly tested for negative values. All critical soil parameters listed above showed the same number of negative entries (5,480), and a cross-check of row indices confirmed that these negative values occurred in the *same set of rows*. These records were therefore classified as globally invalid soil observations.

To assess whether these invalid records were spatially localized or randomly distributed, their geographic distribution was visualized. The soil dataset was converted into a GeoDataFrame using longitude and latitude coordinates, and a spatial scatter map was generated. Valid observations were displayed in light gray, while invalid observations were highlighted in blue (Figure 3.4).

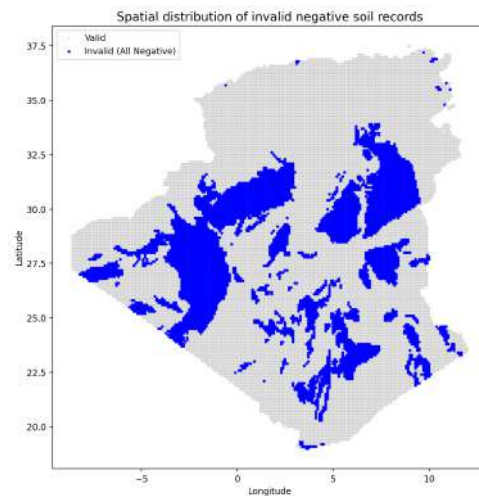


Figure 3.4: Spatial distribution of invalid soil records with negative values. Valid points are shown in light gray, while invalid points (all key soil properties negative) are highlighted in blue.

Solution: Nearest-Neighbor Imputation of Invalid Values

Rather than discarding more than five thousand soil records—which would have introduced spatial gaps and potential bias—a nearest-neighbor imputation strategy was adopted. The objective was to replace invalid values with representative estimates derived from geographically nearby valid observations. For each invalid record, the median value of its neighboring valid points was used for imputation. The median was preferred over the mean due to its robustness to local variability and residual outliers, while still preserving local soil characteristics.

Imputation Procedure

The imputation workflow was implemented as follows:

1. **Identification of Invalid Records:** A boolean mask (`is_invalid`) was created to identify rows where all targeted physical soil attributes were negative.
2. **Preparation of Spatial Data:** All coordinates were ensured to be expressed in WGS84 latitude/longitude and converted to radians, as required for great-circle distance calculations.
3. **Construction of a BallTree:** A BallTree data structure was built using only valid soil records, with the haversine distance metric see Annex A.6. This structure enables efficient spatial nearest-neighbor searches.
4. **Neighbor Querying:** For each invalid record, the 8 nearest valid neighbors within a 100 km radius were queried. If no valid neighbors were found within this radius, the 8 nearest neighbors were retrieved regardless of distance.

5. **Median-Based Imputation:** For each numeric soil attribute, the median of the neighboring valid values was computed and used to replace the invalid negative value. As all imputed attributes were numeric, no categorical imputation was required.
6. **Dataset Update:** The DataFrame was updated with the imputed values, and an audit check confirmed that no records remained with all-negative soil attributes.

Post-Imputation Validation

After imputation, **all 5,480 previously invalid records were corrected**. We verified that no record now has all negative values in the key soil columns (the count of fully-invalid rows dropped to 0). We also generated a new spatial plot: the blue points indicating invalid records have disappeared, confirming that our imputation filled in plausible values for those locations. The dataset's overall quality improved, as every soil property value is now within a realistic range. In summary, we detected a large batch of outlier records characterized by impossible negative soil property values. By leveraging spatial nearest-neighbor imputation (with a BallTree for efficient lookup), we replaced those erroneous values with median values from nearby valid data points. This method preserved the spatial trends and continuity of the soil properties while eliminating clearly invalid entries, resulting in a cleaner and more reliable dataset for downstream analysis or modeling.

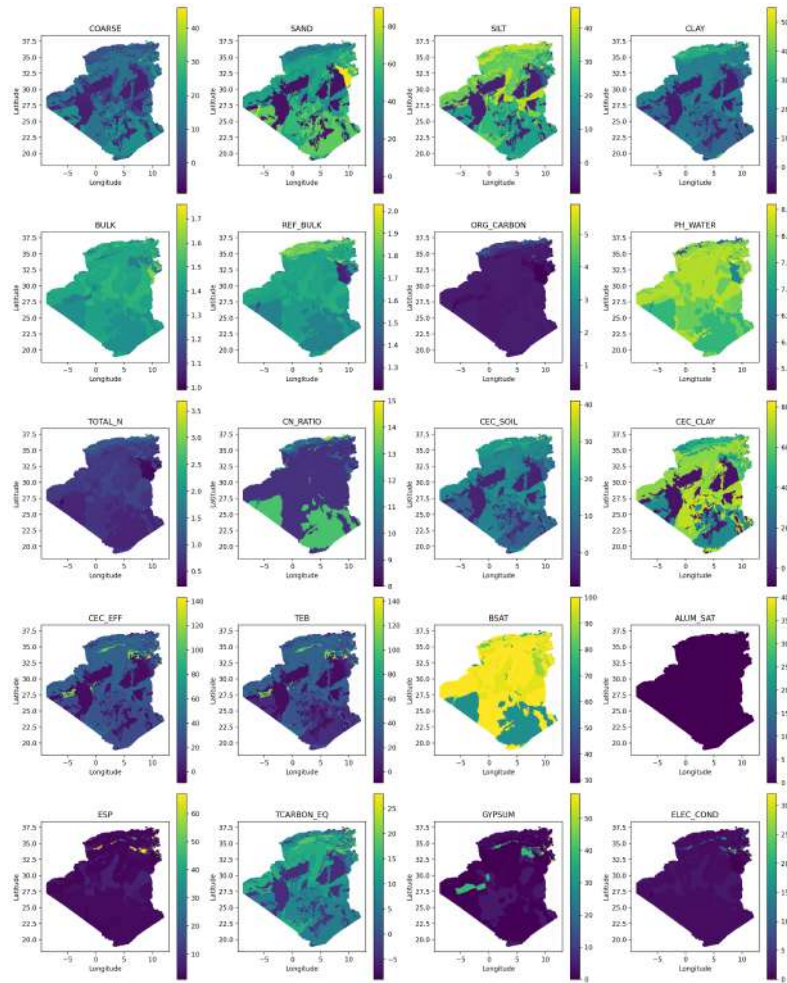


Figure 3.5: Histograms of soil attributes after nearest-neighbor imputation, showing consistent and physically plausible distributions.

The resulting soil property distributions after imputation are shown in Annex A.6, confirming that all variables now exhibit realistic ranges and shapes without negative values.

Detected Statistical Outliers

Using an interquartile-range (IQR)-based detector, the following number of outliers were identified after imputation:

Variable	Outliers	Variable	Outliers
COARSE	28	BSAT	11
SAND	12	ALUM_SAT	28
SILT	0	ESP	4095
CLAY	53	TCARBON_EQ	0
BULK	341	GYP SUM	1182
REF_BULK	969	ELEC_COND	5973
ORG_CARBON	2897	CEC_SOIL	53
PH_WATER	272	CEC_CLAY	0
TOTAL_N	190	CEC_EFF	506
CN_RATIO	1	TEB	506

Table 3.1: Count of statistical outliers detected using the IQR-based method after imputation.

The high outlier counts observed in salinity- and chemistry-related variables (ESP, ELEC_COND, GYP SUM, ORG_CARBON) are consistent with arid and semi-arid soil environments, where strong spatial gradients and extreme values are common.

Case Focus: REF_BULK (Reference Bulk Density)

REF_BULK represents the expected soil bulk density (g/cm^3) derived from pedotransfer functions. Typical mineral soils range between 1.2 and 1.9 g/cm^3 . Values exceeding 2.0 often indicate compaction, erosion, or crusted soils, while very low values suggest organic-rich or loose substrates. REF_BULK is a key indicator of soil structure degradation, which can indirectly increase fire susceptibility by limiting infiltration and inducing vegetation stress USDA Natural Resources Conservation Service [2022].

The Quantiles plot of REF_BULK after imputation shows a near-linear central trend with mild tail deviations, confirming that extreme values are limited and physically interpretable rather than erroneous.

Distributional Analysis by Variable Groups

Texture and Particle-Size Variables COARSE, SAND, SILT, and CLAY exhibit multimodal distributions caused by mixed soil texture classes. None follow a normal distribution due to compositional constraints (percentages summing to 100). These variables are better handled through scaling or compositional transformations rather than outlier removal.

Density and Base Properties BULK and REF_BULK show near-normal distributions, whereas PH_WATER is clearly bimodal, reflecting acidic and alkaline soil regimes. PH_WATER is therefore not amenable to normalization via standard transformations and is better treated using stratified or non-parametric approaches.

Organic Matter and Nutrients TOTAL_N approximates normality. ORG_CARBON, CN_RATIO, and TCARBON_EQ are strongly skewed and multimodal, reflecting varying organic and inorganic carbon pools across ecosystems.

Cation Exchange and Fertility Indicators CEC-related variables and TEB show positive skewness and multi-peaked structures, while BSAT is bounded between 0 and 100, resulting in truncated distributions.

Salinity and Saturation Indicators ALUM_SAT, ESP, GYPSUM, and ELEC_COND are heavily right-skewed with extreme upper tails. These extremes correspond to saline or sodic soils and are ecologically meaningful rather than erroneous.

Methodological Decision

Importantly, no additional outliers were removed at this stage. Since the detected values are physically plausible and reflect genuine soil variability, removing them could bias the dataset and weaken spatial generalization. Instead, all numeric variables will be handled at later stages using normalization or robust scaling techniques appropriate for machine learning, ensuring model stability without altering the underlying data distribution Wilson and Gallant [2021].

3.2.3 Initiation to Feature Engineering: Identification of Redundant Variables

Feature engineering begins with a careful examination of variable dependencies in order to reduce redundancy, prevent multicollinearity, and preserve only informative predictors. In this study, several soil attributes were identified as deterministic or quasi-deterministic transformations of other variables. These variables were therefore evaluated for potential removal prior to normalization and modeling.

Redundancy in Soil Texture Variables

The dataset includes two categorical soil texture indicators:

- TEXTURE_USDA, with unique values {3, 5, 9, 10, 11, 12}
- TEXTURE_SOTER, with unique values {M, C, F, -}

Parallel coordinates plots (Figure 3.6) were used to analyze the relationship between these texture classes and the particle-size fractions SAND, SILT, and CLAY. When grouped by TEXTURE_USDA, the trajectories corresponding to each class are clearly separated and internally consistent: each USDA texture category maps to a unique combination of sand, silt, and clay proportions. This confirms that TEXTURE_USDA is fully determined by these three quantitative variables, as defined by the USDA soil texture triangle.

A similar pattern is observed for TEXTURE_SOTER. Coarse soils (C) exhibit high sand and low clay contents, fine soils (F) show the opposite behavior, and medium soils (M) occupy intermediate ranges. These groupings align directly with the particle-size composition, indicating that TEXTURE_SOTER is likewise a categorical encoding derived from SAND, SILT, and CLAY.

Because both texture variables are deterministic functions of the same base fractions, they do not contribute additional independent information. Retaining them would introduce redundancy and multicollinearity. Consequently, both TEXTURE_USDA and TEXTURE_SOTER were removed from the feature set.

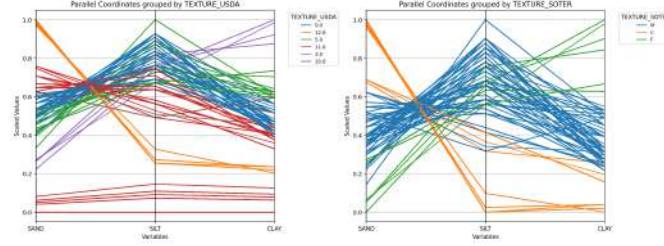


Figure 3.6: Parallel coordinates of particle-size fractions grouped by `TEXTURE_USDA` (left) and `TEXTURE_SOTER` (right), showing that both texture classes are deterministic encodings of sand, silt, and clay composition.

Redundancy of the C/N Ratio

The carbon-to-nitrogen ratio (`CN_RATIO`) is defined in the FAO Harmonized World Soil Database as:

$$\text{CN_RATIO} = \frac{\text{ORG_CARBON}}{\text{TOTAL_N}}$$

To verify this relationship empirically, the provided `CN_RATIO` values were compared with a recomputed ratio obtained directly from `ORG_CARBON` divided by `TOTAL_N`. The scatter and Q-Q plots (Figure 3.7) reveal a near one-to-one correspondence. The Pearson correlation coefficient reaches 0.92 ($p < 0.001$), and the Spearman rank correlation is 0.79 ($p < 0.001$), confirming a strong deterministic dependency.

Minor deviations from the identity line are attributable to rounding effects and numerical precision in the original dataset. Since `CN_RATIO` is a direct transformation of two already available variables, it does not represent an independent soil property. To avoid redundancy, `CN_RATIO` was therefore removed, while `ORG_CARBON` and `TOTAL_N` were retained.

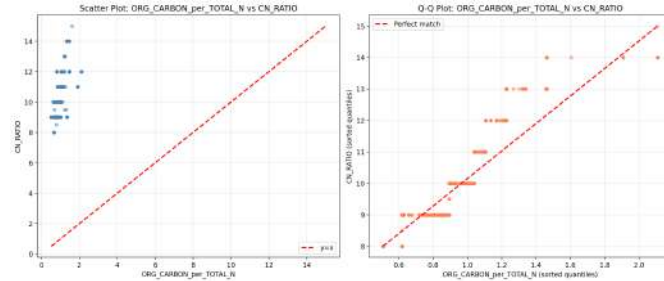


Figure 3.7: Scatter and Q-Q plots comparing `CN_RATIO` to the recomputed ratio `ORG_CARBON / TOTAL_N`, confirming a near-deterministic relationship.

Relationship Between `CEC_SOIL` and `CEC_CLAY`

The Cation Exchange Capacity of clay (`CEC_CLAY`) is theoretically approximated as:

$$\text{CEC}_{\text{clay}} = \frac{\text{CEC}_{\text{soil}}}{\text{Clay fraction}}$$

as described in pedological literature and FAO documentation.

However, empirical evaluation shows that this theoretical approximation does not hold strictly in practice. The computed ratio `CEC_SOIL_per_CLAY` exhibits only a weak-to-moderate relationship with the observed `CEC_CLAY` (Pearson $r = -0.37$, Spearman $\rho = -0.15$, both $p < 0.001$). The scatter and Q-Q plots (Figure 3.8) show substantial dispersion around the 1:1 line.

This variability reflects the influence of additional factors such as organic matter, silt content, and oxide coatings, which also contribute to soil cation exchange capacity. Therefore, despite their conceptual linkage, `CEC_SOIL` and `CEC_CLAY` are not redundant. Both variables were retained as complementary indicators of soil chemical behavior.

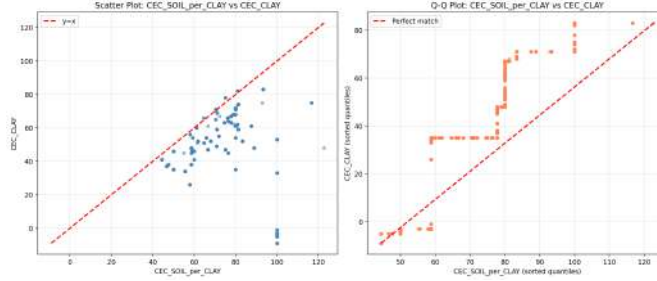


Figure 3.8: Scatter and Q-Q plots comparing `CEC_SOIL_per_CLAY` with observed `CEC_CLAY`, showing substantial deviation from a one-to-one relationship.

Redundancy Between `TEB`, `CEC_EFF`, and `BSAT`

Base saturation (`BSAT`) represents the proportion of a soil's cation exchange capacity occupied by base cations and is defined as:

$$\text{BSAT}(\%) = \frac{\text{TEB}}{\text{CEC_EFF}} \times 100$$

where `TEB` is the total exchangeable bases and `CEC_EFF` is the effective cation exchange capacity [2008]. Comparison of `BSAT` with the computed ratio `TEB_per_CEC_EFF` shows a strong association (Pearson $r = 0.82$, $p < 0.001$); however, diagnostic plots indicate that Pearson's assumptions are violated. The scatter plot reveals a clear non-linear pattern, while the Q-Q plot exhibits a pronounced S-shape, consistent with heavy-tailed distributions and the presence of influential outliers. Given the deterministic ratio structure of `BSAT` and the observed deviations from linearity and normality, Pearson correlation was deemed inappropriate for inference. Rank-based and graphical evidence confirms that `BSAT` is primarily a monotonic but non-linear transformation of `TEB` and `CEC_EFF`, with residual variability arising from measurement uncertainty and rounding effects, particularly near saturation values approaching 100. Accordingly, `BSAT` was removed from the feature set to avoid redundancy and potential multicollinearity issues during modeling, while `TEB` and `CEC_EFF` were retained as the primary indicators of soil chemical fertility and base saturation status.

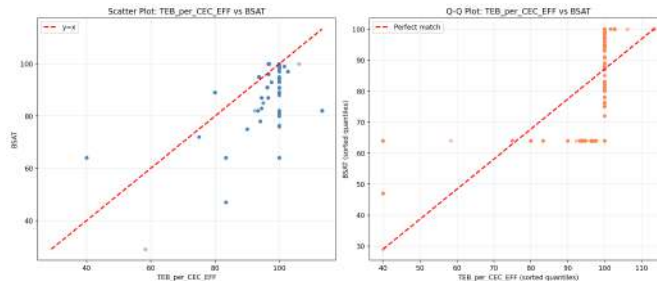


Figure 3.9: Scatter and Q-Q plots illustrating the strong dependency between `BSAT` and the computed ratio `TEB / CEC_EFF`.

Summary of Feature Removal Decisions

Based on theoretical definitions and empirical validation:

- TEXTURE_USDA and TEXTURE_SOTER were removed (derived from particle-size fractions).
- CN_RATIO was removed (directly computed from ORG_CARBON and TOTAL_N).

All remaining variables were retained for subsequent normalization and modeling, ensuring minimal redundancy while preserving the physical interpretability of soil properties.

3.3 Climate Data Cleaning and Missing Value Treatment

3.3.1 Identification of Missing Values

All three climate datasets contain a small but non-negligible proportion of missing values:

- Precipitation: 0.28% missing,
- Maximum temperature: 0.28% missing,
- Minimum temperature: 0.28% missing.

Crucially, missing entries occur on the *same spatial cells* across all three variables, as verified by identical `cell_id` sets. Spatial visualization of missing precipitation values (Figure 3.10) shows that these gaps are concentrated along the perimeter of the Area of Interest, indicating edge effects caused by raster clipping and grid intersection rather than true data absence.

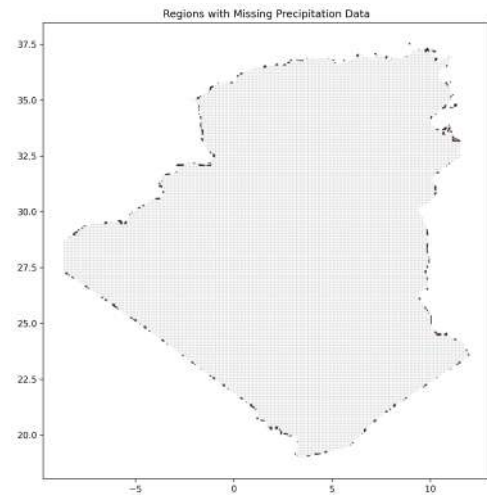


Figure 3.10: Spatial distribution of missing precipitation values, concentrated along the AOI boundaries.

3.3.2 Imputation Strategy

To address missing values while preserving spatial coherence, a two-stage imputation strategy was adopted:

1. **Primary KNN imputation:** Missing values were first imputed using a k-nearest neighbors approach based on spatial proximity, successfully filling approximately 77% of missing entries.
2. **Monthly climatology fallback:** Remaining missing values were imputed using the long-term monthly mean computed over all years for the corresponding variable and month.

This approach ensures that local spatial structure is preserved whenever possible, while guaranteeing complete coverage when neighborhood-based imputation fails (e.g., at extreme boundaries).

After imputation, all missing values were fully resolved:

$$\text{Precip missing} = 0, \quad \text{Tmax missing} = 0, \quad \text{Tmin missing} = 0.$$

3.4 Land Cover Data Cleaning and Feature Preparation

3.4.1 Classification Code Harmonization

The raw `lcccode` column contains complex codes (e.g., 7001 // 8001) which represent mixed, uncertain, or transitional land cover types. The separator (`//`) indicates a mixture between the

two classes. To handle this complexity for machine learning, the following steps were taken:

Legend Integration The official legend files (`globcover_LCCS_legend_africa.xls` and `globcover_legend.xls`) were loaded to establish a mapping between the numeric codes and descriptive labels.

- The `LCCCode` was mapped to its descriptive label (`LCCOwnLabel`) using a dictionary.
- This process identified that certain codes were designated as “Error” in the `LC` column of the legend, yet represented valid land cover types in the `LCCOwnLabel` column (e.g., “Closed forest”). This confirmed that these codes should be retained and mapped correctly.

High-Level Mapping Given the 22 unique `lcccode` values in the dataset, a high-level, simplified mapping was created to aggregate these codes into fundamental categories (e.g., “Forests,” “Bare lands,” “Croplands,” “Vegetation,” etc.). For instance:

- Multiple codes corresponding to water-related classes (e.g., 7001, 8001) were grouped as “Water bodies”.
- Forest-related codes (e.g., 21497-121340) were grouped as “Forests”.

This high-level categorization reduces the number of classes and improves model interpretability, while preserving the main land cover distinctions.

Table 3.2: Example of Landcover Transformation from `lcccode` to `lcc_highlevel`

<code>lcccode</code>	<code>lcc_highlevel</code>
7001 // 8001	Water bodies
0011	Bare Lands
11498	Croplands
21497-121340	Forests
11490 // 11494	Croplands

3.5 Fire Feature Preparation

During this step, only the fire records corresponding to **fire type = 0**, which represent **forest fires**, were retained. All other fire types were removed from the dataset to ensure that the analysis focuses exclusively on forest fire events.

After filtering, a binary feature named **Fire** was generated. Since the dataset now contains only forest fire occurrences, the value of this variable is set to 1 for all remaining records, indicating the presence of a fire.

Finally, to prepare the dataset for integration with other data sources, only the three attributes required for the merge operation were kept, namely **Latitude** and **Longitude**. All other columns were discarded in order to simplify the dataset and reduce unnecessary information.

CHAPTER 4

DATA INTEGRATION AND PREPROCESSING

4.1 Overview

In this chapter, we detail the data cleaning, integration, and preprocessing steps applied before model development. The objective is to construct a consistent, spatially aligned dataset by merging heterogeneous data sources and preparing meaningful features for subsequent analysis.

4.2 Data Merging Procedure

All datasets were spatially harmonized using a common grid resolution of $0.1^\circ \times 0.1^\circ$. The merging process was performed incrementally, combining datasets two by two based on spatial correspondence.

First, the fire dataset was merged with the land cover dataset using a spatial join based on geographic location. Each fire detection point was assigned to a grid cell represented as a polygon. A match was established when the fire point fell inside the corresponding land cover polygon.

Subsequently, additional environmental datasets, including elevation, climate, and soil data, were merged using the same grid-cell polygons as the spatial reference. Since these datasets were already represented at the grid level, the merging was performed directly through polygon-based matching rather than point-in-polygon operations.

This polygon-based integration ensured spatial consistency across all variables and allowed all datasets to be aligned to a unified spatial resolution, facilitating subsequent feature engineering and modeling.

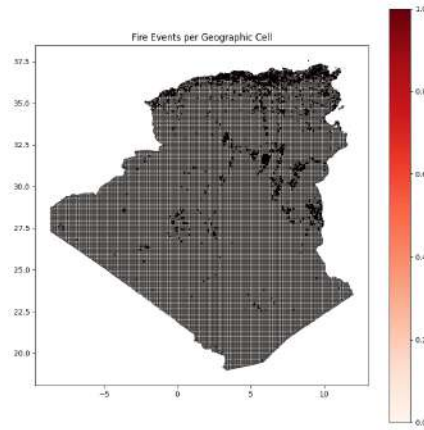


Figure 4.1: Distribution of fire events aggregated per $0.1^\circ \times 0.1^\circ$ geographical grid cell

- **Centroid-based merging:** Climate and soil polygons were matched using representative points (centroids). Exact matches were prioritized, and unmatched climate points were linked to the nearest soil polygon.
- **Visualization:** Coverage maps for climate and soil polygons, as well as elevation distributions and spatial maps, were generated to assess data completeness and spatial consistency.

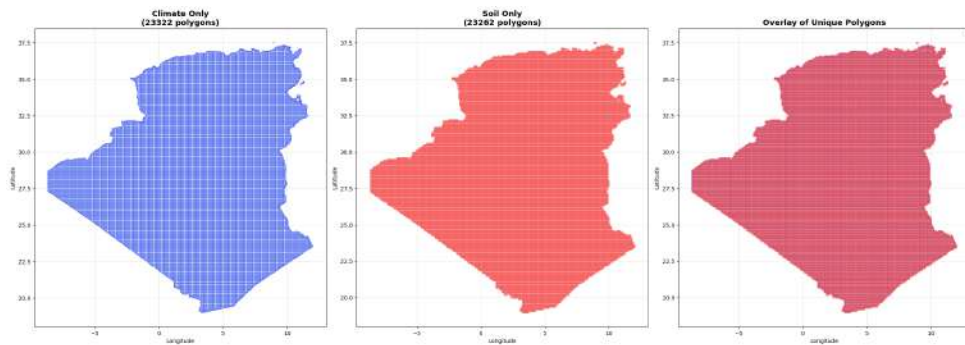


Figure 4.2: Coverage of climate (blue) and soil (red) polygons. Coverage of merged (pink)

4.3 Splitting and Resampling techniques

4.3.1 Splitting to training test and validation set

The dataset was divided into three disjoint sets to ensure robust model evaluation and prevent data leakage. First, the target variable (fire) was separated from the features, and a stratified 70-30 split was applied to create a training set (70%) and a temporary set (30%). Subsequently, the temporary set was further divided equally into validation (15%) and test sets (15%) using the same stratification strategy. This approach ensures that each set maintains the original class distribution, which is particularly important given the class imbalance present in the wildfire data. The Resampling Strategies for Class Imbalance were explained in Annex A.7.

Motivation: Class Imbalance in Wildfire Prediction

The wildfire occurrence variable exhibits a strong class imbalance, with non-fire observations largely dominating fire events. This imbalance is typical in environmental risk modeling and can severely bias supervised learning algorithms toward the majority class, resulting in poor detection of rare but critical events such as wildfires.

To address this issue, several resampling techniques were applied *exclusively to the training set*. Validation and test sets were intentionally left unchanged in order to preserve an unbiased evaluation of model generalization. The objective of resampling is to improve the model's sensitivity to fire occurrences while maintaining meaningful decision boundaries.

Figure 4.3 summarizes the class distributions obtained using the different resampling strategies.

For subsequent modeling experiments, all four versions of the training data were retained to allow a systematic comparison of model sensitivity to resampling choices.

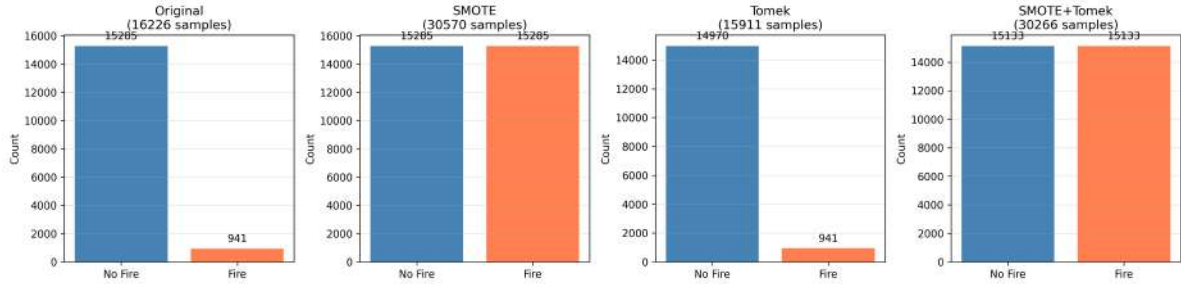


Figure 4.3: Comparison of class distributions in the training set before and after resampling. From left to right: original distribution, SMOTE over-sampling, Tomek Links under-sampling, and combined SMOTE+Tomek.

4.4 Preprocessing and Normalization

4.4.1 Soil Properties:

Soil variables were normalized using property-specific transformations:

- **Texture (COARSE, SAND, SILT, CLAY):** Min-Max scaling to range [0,1].
- **Density and pH:** Robust scaling for densities, Min-Max for pH.
- **Organic Matter and Nutrients:** Log-transform followed by robust scaling for organic carbon; log + standard scaling for total nitrogen; Min-Max for total carbon equivalents.
- **Cation Exchange and Fertility:** Robust scaling for CEC_SOIL, CEC_EFF, TEB; Min-Max for CEC_CLAY, BSAT.
- **Salinity and Saturation:** Log-transform followed by robust scaling for ALUM_SAT, ESP, GYPSUM, ELEC_COND.

4.4.2 Visualization of Normalization Effects

The distributions of selected soil properties before and after normalization were compared to ensure consistency and scaling effectiveness.

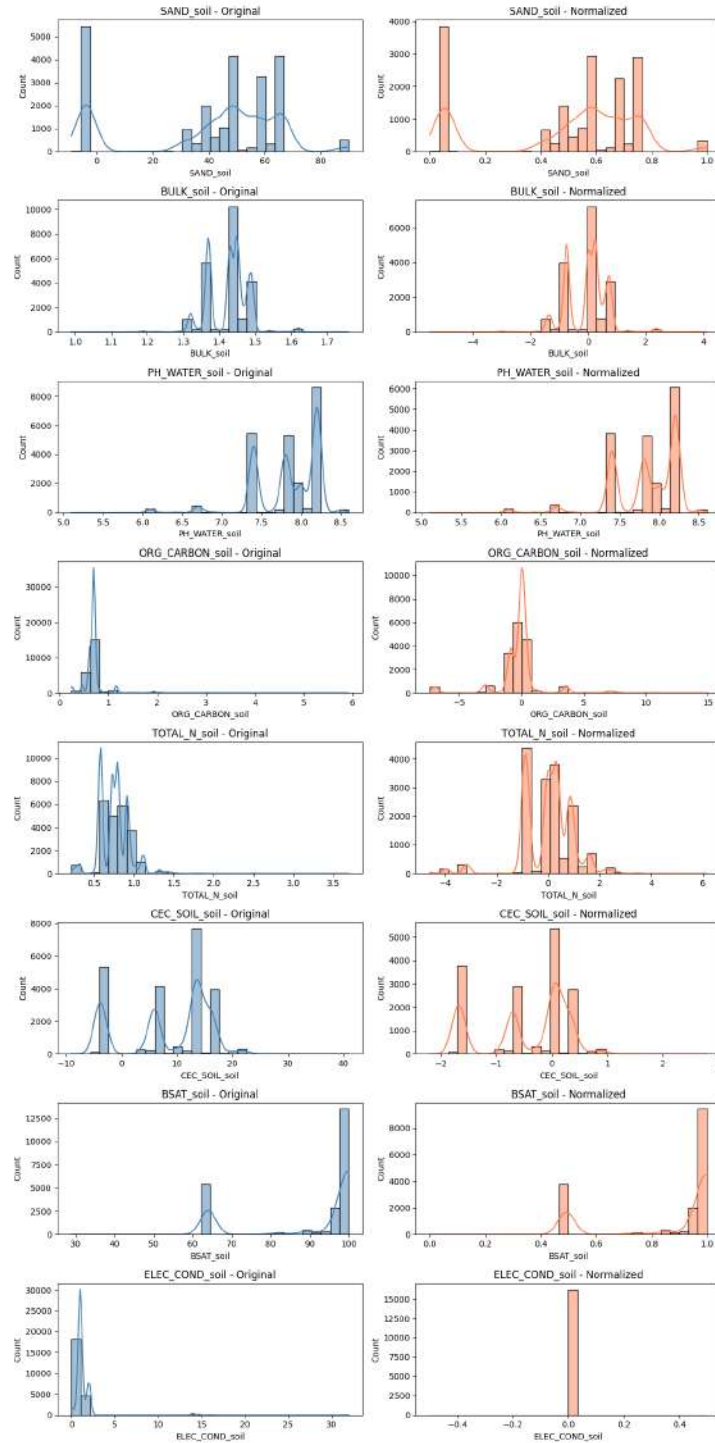


Figure 4.4: Comparison of soil property distributions before and after normalization.

4.4.3 Climate Features:

4.4.4 Expansion of Monthly Climate Variables

Each monthly climate attribute (`month_1` to `month_12`) originally encoded three distinct measurements within a single column: minimum temperature, maximum temperature, and precipitation. To make these data usable by machine learning models, each monthly column was systematically decomposed into three explicit numerical variables.

For each month i , the original triplet was expanded as:

$$\text{month_i} \rightarrow (\text{temp_min_i}, \text{temp_max_i}, \text{precip_i})$$

This transformation converts the nested representation into a fixed-length feature space while preserving seasonal information. The original `month_i` columns were then removed to avoid ambiguity and redundancy. As a result, the climate block consists of 36 interpretable numerical variables (12 months $\times$ 3 indicators), fully compatible with downstream statistical analysis and machine learning models.

Normalisation

Temperature features were standardized (z-score), while precipitation features were log-transformed.

4.4.5 Elevation Features:

Standard scaling was applied to the elevation variable.

4.4.6 Saving the Normalized Data

The final normalized dataset (`X_train`) was saved in Parquet format for subsequent modeling steps.

5.1 Overview

This chapter describes the feature engineering steps applied to the dataset in order to improve its suitability for modeling. These steps include categorical encoding, feature selection, and the removal of redundant attributes introduced during data preparation and merging.

5.2 One-Hot Encoding of Land Cover (High-Level Classes)

The previous mapping resulted in six main land cover classes: **Artificial surfaces**, **Bare lands**, **Water bodies**, **Vegetation**, **Croplands**, and **Forests**.

Following this consolidation, the high-level land cover attribute was transformed using one-hot encoding. This process generated six binary variables, each corresponding to one of the identified land cover classes. The use of one-hot encoding allows categorical land cover information to be incorporated into the model without imposing any artificial ordinal relationships between categories.

5.3 Dropping Unnecessary Columns

In addition to redundant attributes previously identified within each individual dataset, the merging step introduced several internal-use and spatial processing columns that are not relevant for the modeling phase.

To simplify the dataset and reduce noise, these columns were removed. The dropped attributes include: `geometry`, `cell_id`, `match_type`, `distance_m`, `area`, `x`, and `y`.

This dimensionality reduction step helps improve model efficiency and reduces the risk of overfitting by eliminating features that do not contribute predictive information. Column reduction was primarily performed prior to data integration in order to avoid unnecessary data expansion and excessive memory usage; however, no strong inter-feature correlations were identified that would justify further reduction after integration, as the datasets contain heterogeneous information and feature types.

6.1 Overview

This chapter presents the supervised machine learning models used in this study for forest fire prediction. It describes the learning algorithms, training strategy, and evaluation methodology applied to the engineered dataset. The goal is to assess and compare the performance of different supervised classifiers using a consistent experimental setup.

The models considered include Decision Tree (DT), k-Nearest Neighbors (KNN), and Random Forest (RF). Each subsection details the model configuration, training process, and evaluation results.

6.2 Decision Tree Results

This section presents the performance analysis of the Decision Tree classifier using a custom scratch implementation. Three resampling strategies were evaluated in order to address class imbalance: SMOTE, Tomek links, and SMOTE-Tomek. All models were trained using the Gini impurity criterion and evaluated on separate training, validation, and test sets.

6.2.1 Scratch Implementation Results

The table below summarizes the accuracy obtained with the scratch Decision Tree using a fixed maximum depth of 5.

Table 6.1: Decision Tree (Scratch) Performance Across Resampling Strategies

Resampling	Training Acc.	Validation Acc.	Test Acc.	Class 1 Behavior
SMOTE	0.8473	0.7958	0.7970	High recall, low precision
Tomek	0.9558	0.9408	0.9500	Higher precision, low recall
SMOTE-Tomek	0.8507	0.7955	0.7970	Recall = 0.84, Precision = 0.20

For the SMOTE-Tomek dataset, the training set was perfectly balanced with 15,133 samples per class. Despite this balance, the model still exhibited a strong bias toward recall for the minority class (Class 1), achieving a recall of 0.84 but a low precision of 0.20. This behavior indicates that while the model successfully detects most rare events, it also generates a significant number of false positives. The relatively close values between training and test accuracy suggest limited overfitting at this depth.

6.2.2 Scikit-Learn Implementation

For comparison purposes, the Decision Tree classifier was also trained using the scikit-learn implementation. The table below reports the training time and F1-scores for each class.

Table 6.2: Scikit-Learn Decision Tree Performance Across Resampling Strategies

Resampling	Training Time (s)	Training Acc.	Validation Acc.	Class 0 F1	Class 1 F1
SMOTE	2.25	0.9374	0.8749	0.93	0.39
Tomek	0.59	0.9681	0.9382	0.97	0.38
SMOTE-Tomek	2.20	0.9422	0.8818	0.93	0.39

Compared to the scratch implementation, the scikit-learn Decision Tree achieved higher overall accuracy and significantly faster training times. However, the F1-score for the minority class remained relatively low across all resampling strategies, confirming that class imbalance remains a challenging aspect of the task even with optimized libraries.

6.3 KNN Model Configuration and Evaluation

For the KNN model, several dataset variants were tested, and the final model was trained on the **normalized data combined with SMOTE+Tomek resampling**. Normalization was essential since KNN is distance-based; without scaling, features with larger numeric ranges would dominate distance computations and bias the model. Hyperparameter tuning was performed using **Bayesian Search**, which allowed efficient exploration of the parameter space and selection of an optimal configuration focused on maximizing the F1-score.

The best parameters obtained are summarized as follows: Algorithm: **kd_tree**, Distance metric: **Manhattan** ($p = 1$), Number of neighbors: $k = 4$, Weights: **distance**. This configuration achieved a **best cross-validation F1-score of 0.9614**. These exact parameters were then reused to train the **from-scratch KNN implementation**, ensuring a fair and consistent comparison with the library-based model.

6.3.1 Model Performance

The trained model was evaluated on both validation and test sets using multiple performance metrics. K-fold cross-validation was not applied at this stage since normalization was fitted exclusively on the training set.

Table 6.3: KNN performance metrics on validation and test sets

Metric	Predefined KNN		From-Scratch KNN	
	Validation	Test	Validation	Test
Accuracy	0.8924	0.8852	0.8873	0.8769
Precision	0.3082	0.2909	0.3047	0.2776
Recall	0.6915	0.6782	0.7413	0.6980
Specificity	0.9048	0.8980	0.8962	0.8879
F1-score	0.4264	0.4071	0.4319	0.3972
MCC	0.4142	0.3939	0.4280	0.3881
ROC-AUC	0.8789	0.8551	0.8852	0.8636

Overall, the results indicate that the KNN model favors **recall and ROC-AUC**, which is appropriate for fire prediction scenarios where failing to detect a fire event is more critical than generating false positives. Additional visualizations of the KNN model performance are provided in Annex A.8.

6.4 Random Forest

6.4.1 Random Forest – From Scratch Implementation

A **Random Forest classifier was implemented from scratch** in order to better understand the internal mechanisms of ensemble learning and to analyze the impact of different resampling strategies on an imbalanced dataset.

The evaluation was conducted on **10 resampled datasets**, using fixed hyperparameters to ensure a fair comparison between resampling methods.

Experimental Setup

All experiments were conducted using fixed hyperparameters: `n_estimators=100`, `max_depth=10`, `min_samples_split=2`, `min_samples_leaf=1`, bootstrap sampling enabled, and all features considered at each split.

Performance Comparison of Resampling Methods

The table below summarizes the performance of the from-scratch Random Forest on the test set.

Table 6.4: Random Forest (From Scratch) – Test Performance Comparison

Method	Accuracy	Precision	Recall	F1-Score	Time (s)
SMOTE	0.8896	0.9447	0.8896	0.9099	5315.15
Tomek Links	0.9517	0.9439	0.9517	0.9455	2283.09
SMOTE + Tomek Links	0.8916	0.9432	0.8916	0.9109	4948.44

6.4.2 Scikit-learn Implementation

Training was performed using default hyperparameters unless specified otherwise, and performance was assessed using validation accuracy and class-specific metrics.

Performance Across Resampling Strategies

Table 6.5: Scikit-Learn Random Forest Global Performance Across Resampling Strategies

Resampling	Training Time (s)	Training Acc.	Validation Acc.
SMOTE	4.61	0.9599	0.9008
Tomek	1.29	0.9745	0.9471
SMOTE-Tomek	4.11	0.9614	0.8973

Similar to the from-scratch implementation, Tomek Links outperformed other resampling methods, achieving the highest validation accuracy (0.9471) and strong class-wise metrics.

7.1 Overview

This chapter presents the unsupervised machine learning methods applied in this study to explore patterns in the forest fire dataset. It describes the clustering algorithms, parameter selection, and evaluation metrics used to analyze the data without predefined labels.

The algorithms considered include CLARANS, DBSCAN, and K-Means. Each subsection details the configuration, training procedure, and clustering results.

7.2 CLARA K-Medoids Clustering

Clustering Large Applications (CLARA) is a memory-efficient variant of K-Medoids designed for large datasets. Unlike traditional PAM (Partitioning Around Medoids), which computes a full distance matrix, CLARA employs a sampling strategy: it repeatedly draws random samples from the dataset, applies PAM to each sample, and selects the configuration with the lowest total cost. This approach significantly reduces computational complexity while maintaining clustering quality, making it particularly suitable for high-dimensional datasets with tens of thousands of observations.

7.2.1 CLARA Scikit-learn Implementation and Results

CLARA was implemented with 5 sampling iterations, each using 2,000 randomly selected observations. For each iteration, PAM was applied to find optimal medoids, and the configuration with the lowest total inertia was retained. The algorithm was then applied to the full datasets using the optimal $k = 2$ configuration (view in annex Table A.4).

Table 7.1: Final CLARA Clustering Results on Full Datasets ($k = 2$)

Dataset	Data Shape	Silhouette Score	ARI	Inertia
SMOTE	(30,570, 64)	0.9710	0.0149	19,599,265.36
Tomek	(15,911, 64)	0.9693	0.1080	11,229,284.35
SMOTE-Tomek	(30,266, 64)	0.9710	0.0152	19,376,137.45

All three datasets achieved exceptional Silhouette Scores (>0.96), confirming highly compact and well-separated clusters. The Tomek dataset exhibited the highest Adjusted Rand Index ($ARI = 0.1080$), suggesting better alignment between discovered clusters and true class labels. This may be attributed to Tomek Links' removal of ambiguous boundary points, resulting in

clearer class separation. The SMOTE and SMOTE-Tomek datasets showed similar performance, with slightly lower ARI values (0.015), indicating that while clusters are internally coherent, they do not perfectly match the original class structure—an expected outcome in unsupervised clustering.

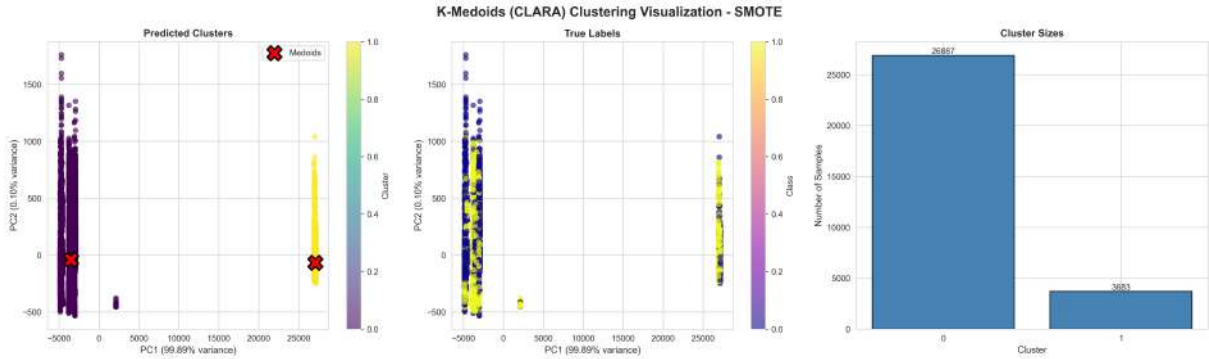


Figure 7.1: CLARA Clustering Visualization for SMOTE Dataset ($k = 2$). Left: Predicted clusters with medoids marked as red crosses. Center: True class labels for comparison. Right: Cluster size distribution showing balanced partitioning.

The PCA projections reveal that CLARA successfully identifies two distinct clusters corresponding to the primary data structure. While the predicted clusters do not perfectly align with the original binary classification (fire vs. no-fire), this discrepancy highlights that the clustering algorithm discovers natural groupings in the feature space rather than reproducing supervised labels.

7.2.2 Bayesian Hyperparameter Optimization

To systematically identify optimal hyperparameters beyond the number of clusters, Bayesian Optimization was employed using Gaussian Process regression. The optimization targeted three key parameters: number of clusters ($k \in [2, 10]$), number of CLARA sampling iterations ($n_{sampling} \in [2, 8]$), and sample size per iteration ($s \in [500, 2000]$). The objective function minimized the negative Silhouette Score over 30 iterations on 3,000-sample subsets.

Table 7.2: Optimal Hyperparameters from Bayesian Optimization

Dataset	Optimal k	$n_{sampling}$	Sample Size	Best Silhouette
SMOTE	2	5	1,100	0.9709
Tomek	2	5	1,100	0.9709
SMOTE-Tomek	2	5	1,100	0.9709

The Bayesian Optimization consistently converged to $k = 2$ clusters across all datasets, validating the initial grid-search findings. Interestingly, the optimizer selected moderate values for sampling iterations (5) and relatively small sample sizes (1,100), suggesting that CLARA achieves stable convergence without requiring exhaustive sampling or large subset sizes. This result underscores CLARA’s efficiency: even with modest computational resources, the algorithm reliably identifies high-quality medoid configurations.

7.2.3 From-Scratch Implementation of CLARANS

CLARANS was implemented from scratch following the parameters: $k = 3$ clusters, $numlocal = 3$ local searches, and a maximum of 200 neighbors per local search. The algorithm uses randomized local search to explore the solution space, replacing one medoid at a time and accepting

configurations that reduce the total cost (sum of distances from points to their nearest medoid). After convergence, cluster labels were assigned and PCA projection to 2D was performed for visualization.

Table 7.3: Results of From-Scratch CLARANS Implementation

Dataset	Shape	Best Cost	Duration (s)	Silhouette	Davies-Bouldin
SMOTE	(30,570, 64)	12,012,065.96	60.49	0.6791	0.3292
Tomek	(15,911, 64)	5,342,093.57	35.50	0.7160	0.2907
SMOTE-Tomek	(30,266, 64)	11,736,406.72	62.85	0.6784	0.3280

The from-scratch implementation successfully converged to stable cluster configurations across all datasets. The Tomek dataset achieved the highest Silhouette Score (0.7160) and lowest Davies-Bouldin Index (0.2907), indicating superior cluster separation and compactness compared to the SMOTE-based datasets. This performance advantage aligns with Tomek Links' methodology of removing noisy boundary samples, resulting in cleaner inter-cluster boundaries that CLARANS can more effectively exploit during medoid selection.

The SMOTE and SMOTE-Tomek datasets produced similar clustering quality (Silhouette ≈ 0.68 , Davies-Bouldin ≈ 0.33), suggesting that the synthetic samples introduced by SMOTE create a more complex feature space where clusters naturally overlap. Despite this increased complexity, both datasets still achieved moderate-to-good cluster separation.

7.3 DBSCAN Model Configuration and Evaluation

DBSCAN experiments were conducted on the **normalized dataset**, as density estimation is entirely distance-driven and highly sensitive to feature scale. Normalization ensures that all variables contribute equally to neighborhood density. Prior to clustering, **PCA was applied with $n_components = 8$** , chosen to preserve the majority of variance while reducing dimensionality, noise, and distance sparsity effects that negatively impact density-based methods.

Hyperparameters were optimized using **Bayesian Search**, leading to the following best configuration:

- $\varepsilon = 0.8207$
- `min_samples = 40`

This optimized configuration increased cluster granularity and separation compared to default settings, while controlling noise. The optimized parameters were then reused for the **from-scratch DBSCAN implementation** to ensure methodological consistency.

7.3.1 DBSCAN Performance Metrics

Table 7.4: DBSCAN metrics comparison across default, optimized, and from-scratch implementations

Metric	Default	Optimized	From-Scratch	
			Validation	Test
ϵ	2.8000	0.8207	-	-
min_samples	15	40	-	-
Number of clusters	5	11	6	5
Noise ratio (%)	0.07	9.21	22.17	22.46
Silhouette score	-0.2686	-0.2268	-0.1811	-0.0038
Davies–Bouldin index	2.3095	2.3344	5.7928	4.4261
Calinski–Harabasz index	360318.48	199812.16	931.19	1133.32
AMI	0.0175	0.0414	0.0293	0.0335
ARI	0.0872	0.0220	-0.0014	0.0054

The results confirm that DBSCAN captures **local density patterns** rather than class separation. Higher noise ratios and low silhouette values are expected given overlapping fire and non-fire regions, while cluster-level fire rates reveal meaningful density-based risk zones despite weak global clustering scores. Visualizations of the clustering results for both implementations are provided in Annex A.12.

7.4 K-Means Model Configuration and Evaluation

K-Means clustering was applied on the **normalized and resampled dataset**, as normalization is essential to prevent features with large numeric ranges from dominating centroid distances, while resampling was used to mitigate class imbalance effects when interpreting clusters with respect to fire occurrence. Prior to clustering, **PCA with $n_components = 2$** was applied to reduce dimensionality while preserving the dominant variance structure and enabling more stable and interpretable cluster separation.

Hyperparameter tuning was performed using **Bayesian Search**, which identified an optimized configuration with a higher number of clusters and improved compactness. The best parameters retained were:

- Number of clusters (K): 8
- Initialization: **k-means++**
- Maximum iterations: 300
- Number of initializations: 20

These parameters were used for both the predefined implementation and the from-scratch K-Means model to ensure consistency.

7.4.1 K-Means Performance Metrics

Table 7.5: K-Means metrics comparison using default and Bayesian-optimized parameters

Metric	Default	Optimized
Number of clusters (K)	3	8
Inertia	2.79×10^9	6.63×10^8
Silhouette score	0.7148	0.5991
Davies–Bouldin index	0.2921	0.4907
Calinski–Harabasz index	2.81×10^6	3.38×10^6
Number of iterations	5	5
Runtime (s)	3.29	0.47

Predefined K-Means (Default vs Optimized)

Table 7.6: From-scratch K-Means metrics on validation and test sets

Metric	Value
Number of clusters (K)	5
Inertia	2.32×10^9
Silhouette score	0.60
Number of iterations	26
Runtime (s)	6.98
Centroids shape	(5, 64)

From-Scratch K-Means Results The optimized predefined K-Means model achieved a significantly lower inertia and faster convergence, while the from-scratch implementation required more iterations to converge but reached comparable silhouette performance. Differences are primarily attributed to centroid initialization strategy and convergence criteria implementation details. Visualizations of the clustering results for both implementations are provided in Annex A.11.

A.1 Soil Data Extraction from HWSD2

This section describes in detail how the soil variables were extracted from the Harmonized World Soil Database v2 (HWSD2) and mapped onto the common 0.1° grid used in this project. The objective is to derive, for each 0.1° cell within the Algeria–Tunisia area of interest (AOI), a set of physically meaningful topsoil properties (0–20 cm) suitable for downstream fire prediction models.

A.1.1 Implementation environment

The extraction is implemented in Python using the following main libraries:

- **NumPy** and **pandas** for numerical arrays and tabular data manipulation.
- **GeoPandas** and **shapely** for vector geometries (reading the AOI, fixing topology, constructing grid-cell polygons).
- **rioxarray** and **rasterio** (via `rasterio.enums.Resampling`) for raster I/O, clipping and resampling of the HWSD2 soil map.
- **pyodbc** to connect to the `HWSD2.mdb` Microsoft Access database.
- **matplotlib** for quick visualisation (e.g. map of pH).

A small utility function ensures that output directories exist before writing files, so that all CSV and GeoPackage outputs are created under the desired folder structure without manual preparation.

A.1.2 Input HWSD2 data and file structure

The HWSD2 dataset is provided in two main components:

- **HWSD2__RASTER/** containing:
 - `HWSD2.bil`: a Binary Interleaved raster whose pixel values are integer *soil mapping unit* codes.
 - `HWSD2.hdr`: an ASCII header specifying raster dimensions (number of rows/columns), data type, byte order, cell size and origin.

- **HWSD2.prj**: the projection (WKT) describing the coordinate reference system (CRS), typically a geographic CRS close to EPSG:4326.
- **HWSD2.stx**: an ancillary statistics or index file; its content is not directly used in this workflow.
- **HWSD2_DB/** containing:
 - **HWSD2.mdb**: a Microsoft Access database which stores the attribute tables describing the soil mapping units and their properties.

The raster **HWSD2.bil** does *not* contain soil properties directly. Instead, each pixel stores an integer code identifying a *Soil Mapping Unit* (SMU). The associated lookup tables in **HWSD2.mdb** link these SMU codes to detailed physical and chemical soil attributes.

From a database perspective, the key tables are:

- **HWSD2_SMU**: one row per soil mapping unit, with a primary key such as **HWSD2_SMU_ID**. This table provides high-level, mapping-unit-level attributes (coverage, dominant texture class, drainage, classification codes, etc.).
- **HWSD2_LAYERS**: up to seven rows per SMU, each corresponding to a depth interval (layers D1, D2, D3, ...). This table stores numeric soil properties per layer, such as sand/silt/clay percentages, organic carbon, pH, bulk density, cation exchange capacity and so on.
- **Dictionary (D_*) tables**: codebooks that map categorical codes in **HWSD2_SMU** and **HWSD2_LAYERS** to human-readable labels. Examples include **D_TEXTURE_USDA**, **D_DRAINAGE**, **D_AWC**, **D_ROOT_DEPTH**, **D_ROOTS**, **D_PHASE**, **D_IL**, **D_SWR**, **D_ADD_PROP**, **D_FA090**, **D_COVERAGE** and **D_KOPPEN**.
- **WRB/FAO classification tables**: e.g. **WRB_Class**, **D_WRB4**, **D_WRB2**, **D_WRB_PHASES**, which link WRB and FAO symbols (**WRB4**, **WRB2**, **WRB_PHASES**, **FA090**) to their class names and sometimes colors.
- **Metadata tables**: **HWSD2_SMU_METADATA** and **HWSD2_LAYERS_METADATA**, which list for each field its name, unit, textual description and, when applicable, the corresponding dictionary table (column **DOMAIN**).

These metadata tables are crucial to interpret the units (e.g. % for **SAND**, pH units for **PH_WATER**) and to know which categorical fields can be decoded via which dictionary table.

A.1.3 Mapping units (MU / SMU) as join keys

In **HWSD2**, each raster pixel holds an integer code representing a soil mapping unit. The terminology varies slightly across versions and documentation (**MU_GLOBAL**, **SMU_ID**, **HWSD2_SMU_ID**, etc.), but the concept is always the same:

- The raster band is a grid where each cell contains a single integer.
- That integer is an identifier for a soil mapping unit, not a physical variable.
- The details (texture, carbon, pH, etc.) are stored in the database and must be looked up using this identifier as a foreign key.

The extraction workflow therefore uses the raster only to determine the mapping unit present in each 0.1° grid cell in the AOI, then uses that ID to retrieve the corresponding topsoil properties from **HWSD2.mdb**.

A.1.4 Construction of a 0.1° mapping-unit grid

The first step is to build a regular $0.1^\circ \times 0.1^\circ$ grid of mapping-unit IDs over the Algeria–Tunisia AOI.

AOI preparation. The AOI is stored as a GeoJSON file (here `AOI_DZA_TUN/aoi_dza_tun.geojson`) and is read as a vector dataset using GeoPandas. If the file does not specify a CRS, it is explicitly set to EPSG:4326 and then reprojected to EPSG:4326 to enforce a consistent geographic coordinate system. Invalid polygons are fixed using a zero-distance buffer, and all AOI geometries are dissolved into a single multi-polygon via a unary union. This geometry is then converted to a simple mapping (GeoJSON-like) structure, as required by the raster clipping functions.

Raster loading and clipping. The soil raster `HWSD2.tif` is opened with a geospatial raster reader (via `rioxarray.open_rasterio`). The associated header and projection files provide the raster dimensions, geotransform and CRS. If no CRS is found at read time, a default EPSG:4326 CRS is written to the raster object, consistent with the AOI. The raster is then clipped to the AOI geometry, dropping pixels outside the study area.

Reprojection and resampling to 0.1° . The clipped raster is resampled onto a regular grid with a target resolution of 0.1° in longitude and latitude. Because the pixel values are categorical mapping-unit IDs, the resampling method must preserve categories rather than interpolate numeric values. For this reason, a *mode* resampler (from `rasterio.enums.Resampling`) is used: for each 0.1° grid cell, the most frequent mapping unit among the original pixels falling into that cell is chosen as the representative value.

The result is a new raster aligned on a regular 0.1° grid in EPSG:4326, where each cell contains a dominant mapping-unit ID. This raster is then converted to a tabular format, with one row per grid cell and columns for the cell centre coordinates (x longitude, y latitude) and the mapping-unit ID. No-data cells are removed, and the mapping-unit values are cast to integer.

For traceability, this intermediate grid of mapping units is exported as a CSV file (here `hwsd2_grid01_debug.csv`) under `ExtractedDatasets/SoilFeatures/` before any database joins, so that the gridding step can be inspected independently.

A.1.5 Extraction of topsoil (D1) properties from the HWSD2 database

Database connection and schema inspection. The `HWSD2.mdb` Access database is opened using a suitable ODBC driver (via `pyodbc`). Connection pooling is disabled to avoid handle reuse issues, and the connection is configured with explicit Unicode encodings (`utf-8` and `utf-16le`) to correctly handle any non-ASCII field names or metadata.

Once connected, a small sample (e.g. five rows) is queried from the `HWSD2_SMU` and `HWSD2_LAYERS` tables to inspect their column names and confirm the actual primary key used for soil mapping units. Because naming conventions vary across releases, the code automatically searches for reasonable key candidates such as `HWSD2_SMU_ID`, `SMU_ID`, `MU_GLOBAL`, `MUKEY` or `MU`. In the present database, the key was detected as `HWSD2_SMU_ID` in both `HWSD2_SMU` and `HWSD2_LAYERS`. The depth dimension is represented by a `LAYER` column, with values `D1`, `D2`, ... corresponding to successive depth intervals.

Choice of depth layer: D1 (0–20 cm). The project focuses on topsoil properties because the 0–20 cm layer is the most relevant for vegetation, root activity, water balance and many fire-related processes. In HWSD2, this topsoil layer is encoded as `LAYER = 'D1'`, which in the documentation corresponds to 0–20 cm depth. Deeper layers (`D2`, `D3`, ...) cover 20–50 cm, 50–100 cm and so on, but are not used in this first version of the dataset.

The extraction therefore filters `HWSD2_LAYERS` to keep only the rows where `LAYER = 'D1'`, effectively selecting one record per mapping unit for the topsoil.

Selection and normalisation of feature columns. A target list of soil variables is defined for modelling. These include, for example:

- Particle-size and texture fields: `COARSE`, `SAND`, `SILT`, `CLAY`, as well as categorical texture classes such as `TEXTURE_USDA` and `TEXTURE_SOTER`.
- Bulk density: `BULK` and `REF_BULK`.
- Carbon and nutrients: `ORG_CARBON`, `TOTAL_N`, `CN_RATIO`.
- pH and exchange capacity: `PH_WATER`, `CEC_SOIL`, `CEC_CLAY`, `CEC_EFF`, `TEB`, `BSAT`, `ALUM_SAT`, `ESP`.
- Additional chemistry: `TCARBON_EQ`, `GYP SUM`, `ELEC_COND`.

Because column names in the MDB can differ from these canonical labels (for example, `BULK_DENSITY` instead of `BULK`), a synonym mapping is implemented: for each desired feature, the code searches in the database schema for exact matches or known variants and builds a mapping from the canonical name to the actual column name. After loading the D1 records, the columns are renamed back to the canonical names so that the rest of the pipeline can treat them consistently.

Numeric fields are then cast to numeric types, coercing invalid or missing entries to `NaN`. To ensure that only one D1 record per mapping unit is present, duplicates on the SMU key are dropped if needed.

Interpreting categorical and classification fields. While the extraction step primarily focuses on numeric fields, the presence of categorical attributes and classification codes is documented for completeness. Fields such as `TEXTURE_USDA`, `DRAINAGE`, `AWC`, `ROOT_DEPTH`, `ROOTS`, `IL`, `PHASE1`, `PHASE2` and `ADD_PROP` can be decoded using the dictionary tables mentioned earlier. Similarly, WRB and FAO classification codes (`WRB4`, `WRB2`, `WRB_PHASES`, `FAO90`) can be mapped to soil class names via `WRB_Class`, `D_WRB4`, `D_WRB2`, `D_WRB_PHASES` and `D_FAO90`. The metadata tables `HWSD2_LAYERS_METADATA` and `HWSD2_SMU_METADATA` indicate, for each field, which dictionary table to use (column `DOMAIN`) and what unit the values are expressed in. This information is essential for later interpretation and potential feature engineering, even if the raw extraction stage only stores the codes.

A.1.6 Join with the 0.1° grid and export

Key harmonisation and merge. On the grid side, the mapping-unit column initially carries a generic name (e.g. `MU_GLOBAL`) and is stored as an integer. On the database side, the SMU key is `HWSD2_SMU_ID`. Before merging, the script harmonises the key names and types: both columns are renamed to a common identifier (for example `HWSD2_SMU_ID`) and converted to a nullable integer type. A left join is then performed between the 0.1° grid table and the D1 layer attributes on this key.

This operation attaches, to each grid cell, the full vector of topsoil properties corresponding to the dominant SMU in that cell. A quick coverage check counts the total number of grid cells and the number of cells with non-missing values for a representative numeric field (e.g. `COARSE`) to verify that almost all cells were successfully matched. In this case, the match rate is effectively 100 %.

Creation of grid-cell polygons and GIS export. For GIS usage and visual validation, each 0.1° cell is converted to a square polygon centred at (x, y) with side length 0.1° (half-size 0.05° in each direction). These polygons, together with the joined attributes, are stored in a GeoDataFrame with CRS EPSG:4326 and written to a GeoPackage file. The same data are also saved as a flat CSV table (here `hwsd2_grid01_D1.csv` under `ExtractedDatasets/SoilFeatures/`) for use in the machine learning pipeline. The script additionally includes an optional export to a Parquet file (commented out in the current run) to allow a more compact binary representation if needed later.

Quality checks and quick visualisation. Basic exploratory statistics (mean, standard deviation, minimum, maximum) are computed for the numeric columns, and the proportion of missing values per variable is reported to detect potential anomalies. A quick map, such as a choropleth of `PH_WATER` over the AOI, is generated from the GeoPackage to visually confirm that the spatial patterns are plausible and that there are no obvious artefacts from clipping or resampling.

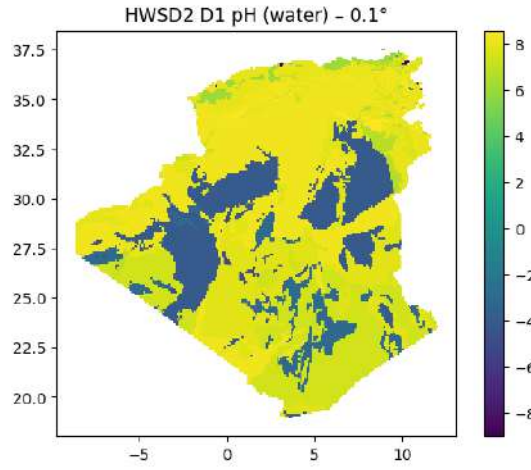


Figure A.1: Spatial distribution of topsoil pH (water) over the AOI at 0.1° resolution.

A.1.7 Summary

In summary, the soil extraction process does not compute new properties from the raster itself. Instead, it uses the raster pixel values as mapping-unit identifiers, aggregates them to a regular 0.1° grid via a mode resampler, and then uses these identifiers as foreign keys to query the HWS2 database for topsoil (D1) attributes. The result is a spatially consistent, GIS-ready and ML-ready dataset of soil properties for each 0.1° cell in the Algeria–Tunisia AOI.

A.2 Normality assessment using quantiles plots

Figure A.2 presents the quantiles plots of the soil variables against a theoretical normal distribution.

COARSE shows points following the reference line reasonably well, with minor tail deviations, indicating slight non-normality. **SAND** displays strong deviation at high quantiles, confirming its positively skewed, non-normal nature. **SILT** exhibits a pronounced S-shaped pattern, with clear departures in both tails, indicating non-normality. **CLAY** aligns closely with the reference line, with only small deviations at the extremes, and can be considered normally distributed.

BULK presents a clear right tail and upward deviation at high quantiles, confirming non-normality. **REF_BULK** closely follows the theoretical line across most quantiles, indicating

an approximately normal distribution. **ORG_CARBON** shows mild curvature at the tails but remains largely aligned, suggesting near-normal behavior. **PH_WATER** shows strong deviations and non-linearity, indicating clear departure from normality.

TOTAL_N exhibits an almost linear pattern with slight deviations at both tails, indicating slight normality. **CN_RATIO** presents an S-shaped curve with upper-tail deviation, confirming non-normality. **CEC_SOIL** aligns well with the reference line, with only minor tail effects, and can be considered normally distributed. **CEC_CLAY** shows slight upward deviation in the upper tail but otherwise follows the line, indicating slight normality.

CEC_EFF, **TEB**, **BSAT**, **ALUM_SAT**, **ESP**, **GYP SUM** and **ELEC_COND** all exhibit strong curvature and substantial deviations from the reference line, confirming pronounced right-skewed and non-normal distributions. **TCARBON_EQ** shows relatively good alignment with mild upper-tail deviation, indicating slight normality.

These results confirm that most soil variables do not follow a Gaussian distribution and will require appropriate transformations or robust scaling prior to statistical modelling.

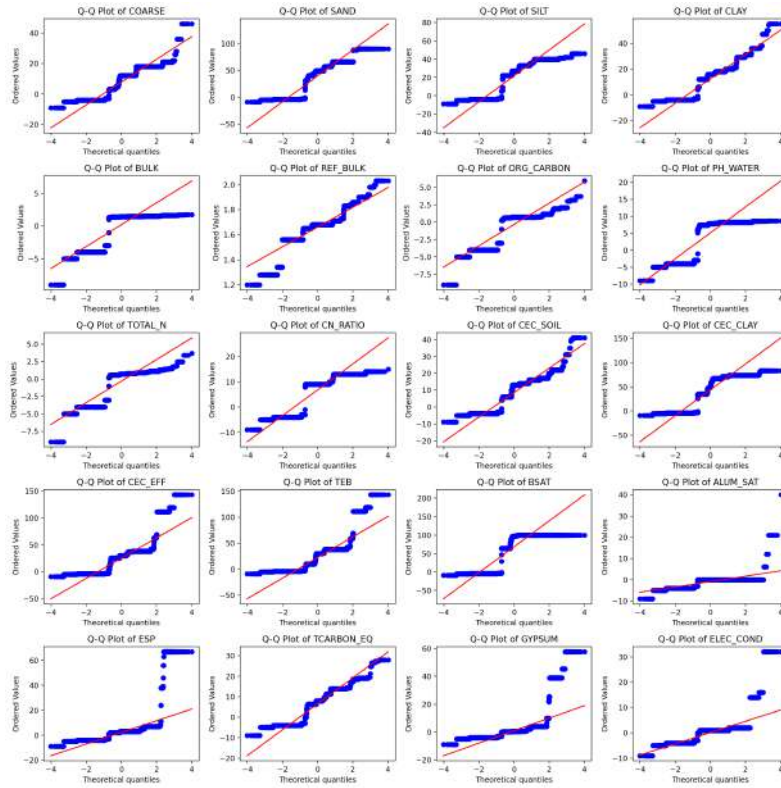


Figure A.2: Quantiles plots of soil numeric variables against the normal distribution.

A.3 Elevation dataset description and Visualisation

A.3.1 Distribution and Topographical Profile

The distribution of all valid elevation values was analyzed to understand the region's topographical profile.

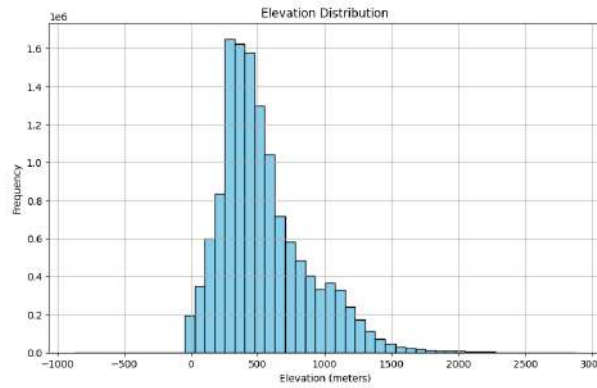


Figure A.3: Frequency distribution of elevation values in the merged Elevation Dataset. The distribution exhibits a clear left skew.

Histogram Interpretation The elevation histogram exhibits a **left-skewed distribution**. This indicates that low-elevation areas dominate the region, while higher elevations occur less frequently, forming a long left tail. The majority of the terrain is therefore concentrated in low to mid-elevation ranges, corresponding to coastal plains and steppe regions, with higher elevations associated with inland mountain ranges.

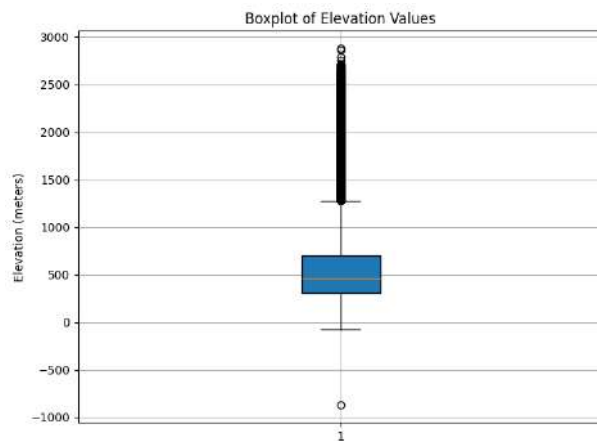


Figure A.4: Boxplot summarizing the elevation data, highlighting the interquartile range (IQR) and the presence of extreme high-elevation outliers.

Boxplot Interpretation The boxplot confirms the presence of significant high-elevation **outliers**. These extreme values correspond to the highest points in the study area, notably the **Tassili n'Ajjer** and the **Hoggar Mountains (Ahaggar)** in southern Algeria, which reach peak elevations of approximately 2100 to 3000 meters.

A.4 Fire data description and visualization

A.4.1 Variable Definitions and Geospatial Metrics

Understanding the physical meaning of the VIIRS variables is critical for interpreting the visualizations. The following table summarizes key columns used in the analysis:

Table A.1: Summary of Key VIIRS Fire Dataset Variables

Variable	Meaning	Unit
bright_ti4	Brightness temp., Band M13 ($4\mu\text{m}$)	Kelvin (K)
bright_ti5	Brightness temp., Band M15 ($11\mu\text{m}$)	Kelvin (K)
frp	Fire Radiative Power	Megawatts (MW)
scan	Pixel size in scan (E-W) direction	Degrees
track	Pixel size in along-track (N-S) direction	Degrees

A.4.2 Exploratory Data Analysis

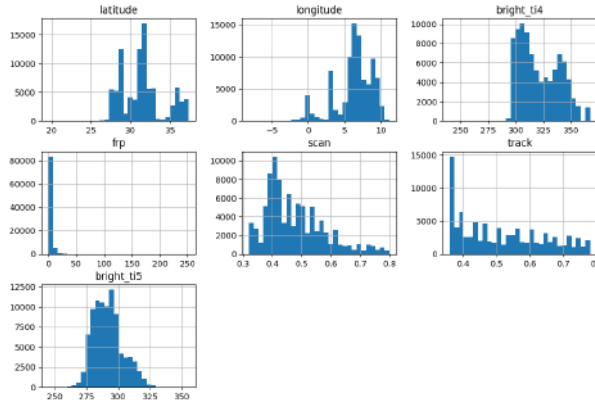


Figure A.5: Histograms illustrating the distribution of key numerical variables (`latitude`, `longitude`, `bright_ti4`, `frp`, `scan`, `track`, `bright_ti5`) in the merged fire dataset.

Feature Distribution and Correlation The distributions of the numerical columns were visualized using histograms, revealing a heavily left-skewed distribution for intensity variable `frp`, indicating that most fire detections are of low power with few extreme events, which confirms what has been shown in the previous scatter plot `frp` map.

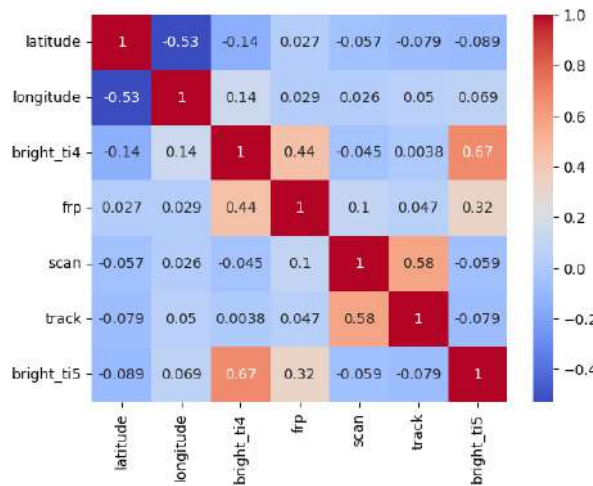


Figure A.6: Heatmap of the Pearson correlation matrix for numerical variables. High correlation exists between the brightness temperatures and the pixel dimensions.

A correlation matrix heatmap was generated to explore relationships between the numerical features. Key findings include:

- A very high correlation (Spearman $\rho \approx 0.96$) was observed between `bright_ti4` and `bright_ti5`, which is expected as both measure thermal radiation from the same source, but in different bands.
- Positive correlations were also noted between `bright_ti4`, `frp`, `scan`, and `track`, suggesting that larger or more intense fires (higher `frp`) often have higher brightness temperatures and are more likely to be detected in larger, more distorted pixels (higher `scan` and `track` values).

A.5 Climate Data Visualization Summary

Spatial Patterns

Figure A.7 illustrates spatial heatmaps of mean climate conditions over the 2020–2024 period.

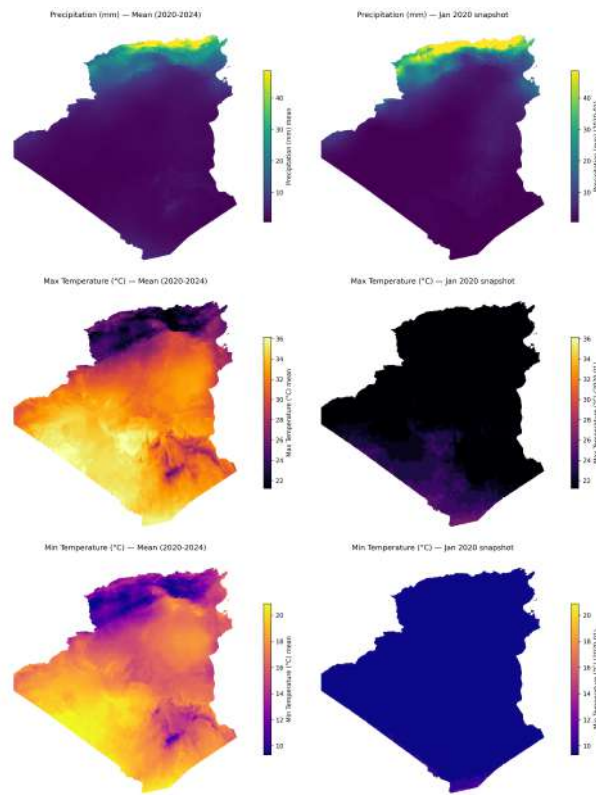


Figure A.7: Spatial heatmaps of mean precipitation, maximum temperature, and minimum temperature (2020–2024).

Inter-variable Relationships

Scatter plots comparing precipitation with minimum and maximum temperatures (Figure A.8) reveal moderate negative correlations ($r = -0.236$ for T_{min} , $r = -0.335$ for T_{max} , $p < 0.001$). Higher precipitation values are associated with lower temperatures, consistent with physical processes such as cloud cover, evaporation, and seasonal rainfall regimes.

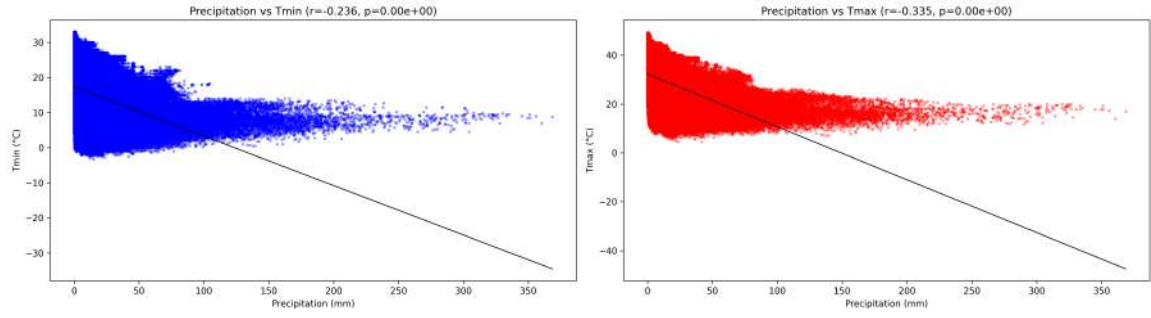


Figure A.8: Relationship between precipitation and temperature variables.

A.6 Soil Data Cleaning Summary

A.6.1 Spatial Visualization of Soil Variables

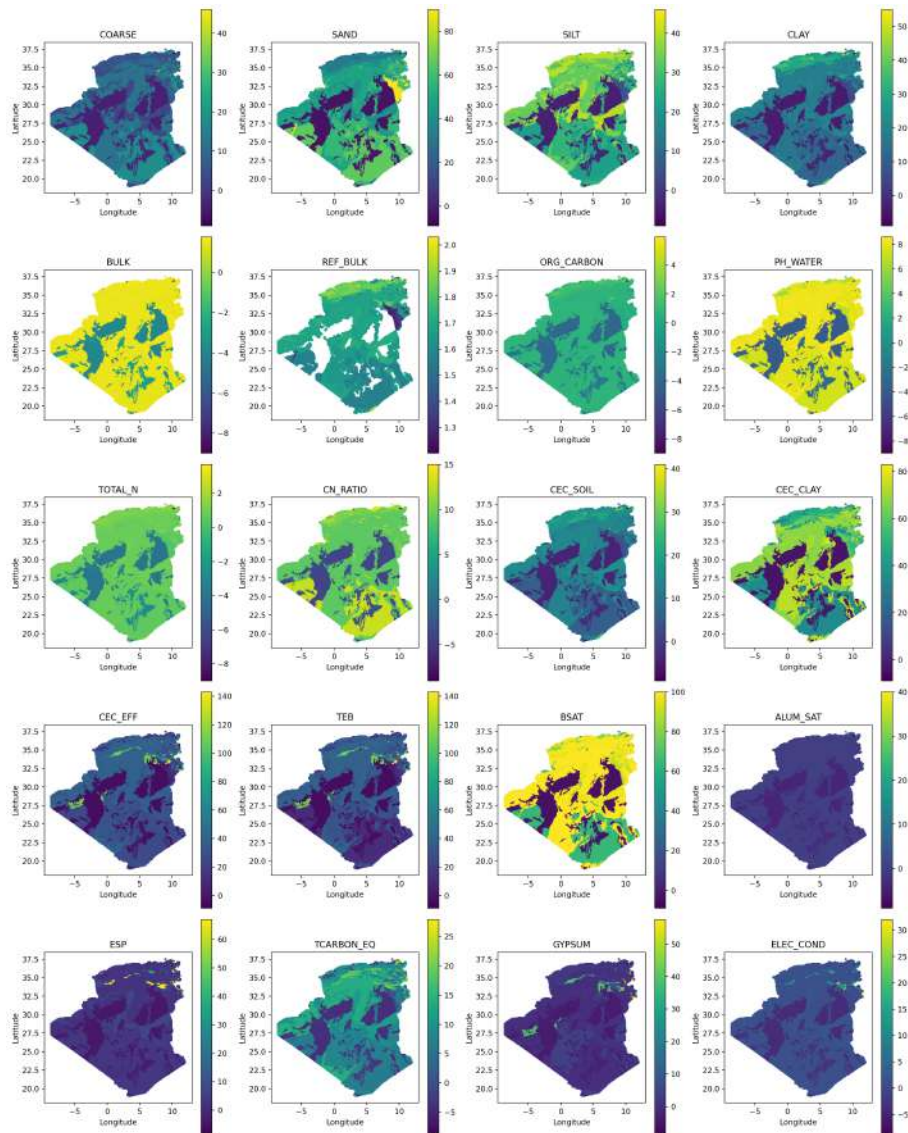


Figure A.9: Spatial variation maps (subplot grid) of the numeric soil variables over the study area.

A.6.2 BallTree Algorithm in the Context of Soil Data

The BallTree algorithm played a central role in enabling efficient and spatially meaningful imputation. A BallTree recursively partitions the data space into nested hyperspherical regions (“balls”), allowing rapid nearest-neighbor searches compared to brute-force approaches Omohundro [1989], scikit-learn developers [2023].

In this study, the use of the haversine metric ensured that distances between points were computed as true great-circle distances on the Earth’s surface, making the notion of “nearest” geographically accurate Sinnott [1984]. This was particularly important for soil properties, which are spatially autocorrelated. Without a spatial indexing structure such as BallTree, querying neighbors for 5,480 invalid records against a large geospatial dataset would have been computationally prohibitive. The BallTree approach therefore provided both efficiency and physical relevance.

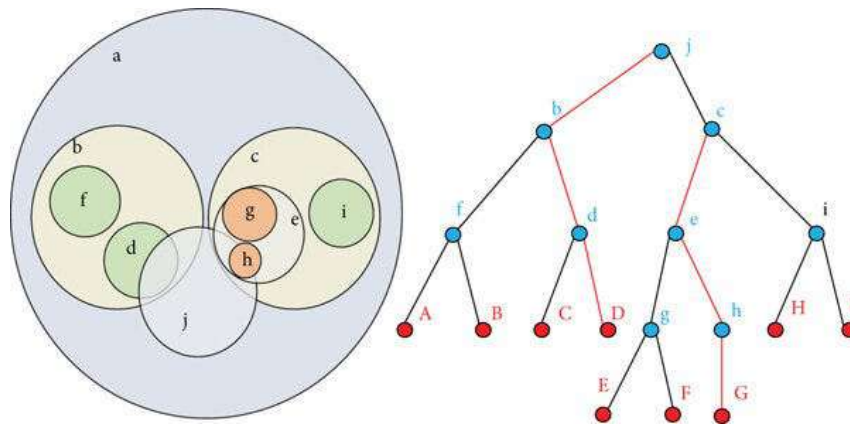


Figure A.10: Conceptual illustration of the BallTree data structure, showing recursive spatial partitioning and efficient nearest-neighbor search.

A.6.3 Residual Outliers After Imputation

After correcting invalid negative records using spatial nearest-neighbor imputation, the soil dataset was re-evaluated for remaining outliers. Although the physically impossible values were successfully eliminated, statistical outliers were still detected across several soil variables. These remaining outliers do not necessarily correspond to data errors, but rather reflect natural soil heterogeneity, extreme environmental conditions, or localized geochemical regimes.

Figure A.11 and Figure A.12 summarize the post-imputation distributions using kernel density estimates (KDE) and Quantiles plots, respectively.

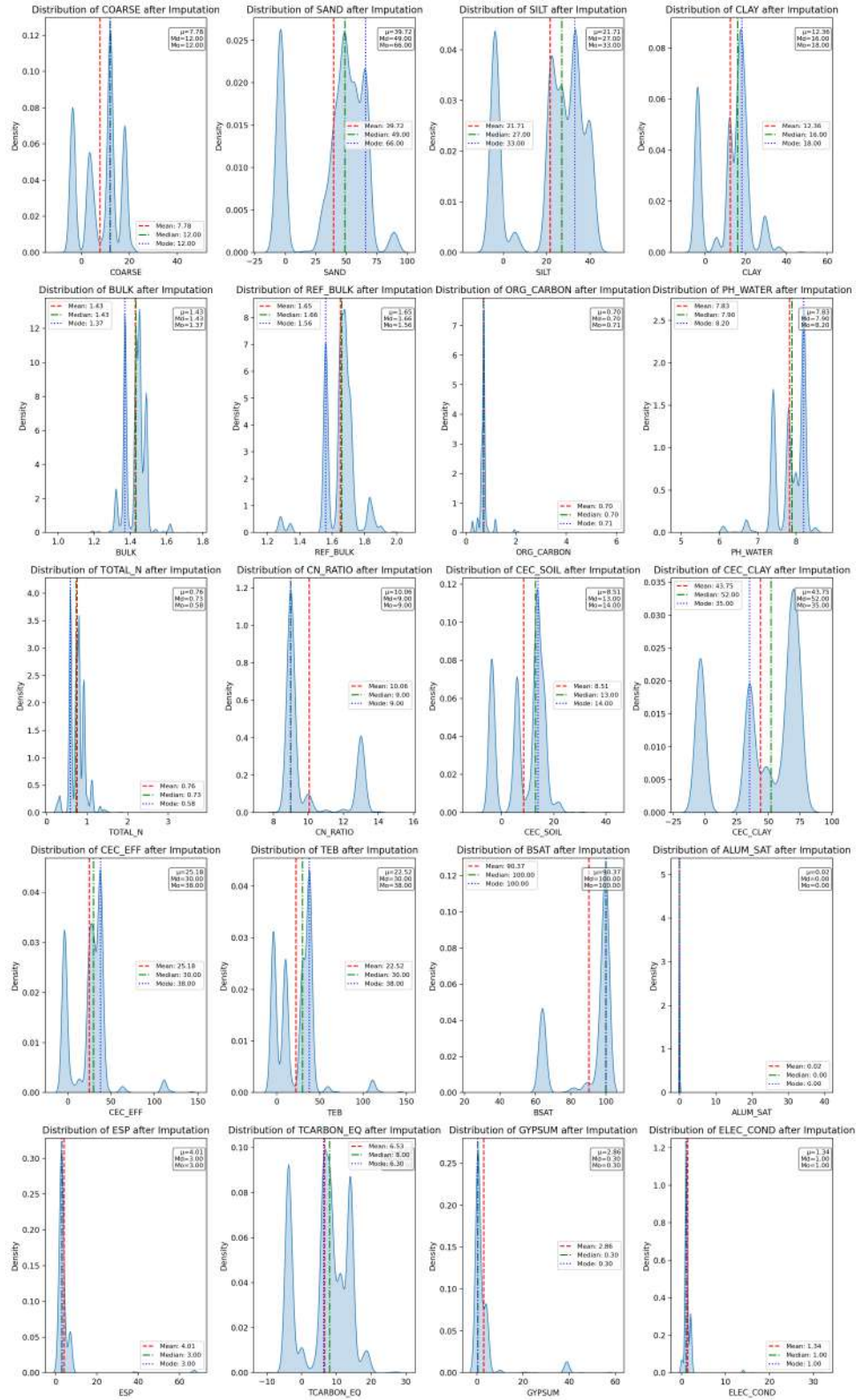


Figure A.11: Kernel density estimates of soil variables after imputation. Vertical lines indicate mean, median, and mode.

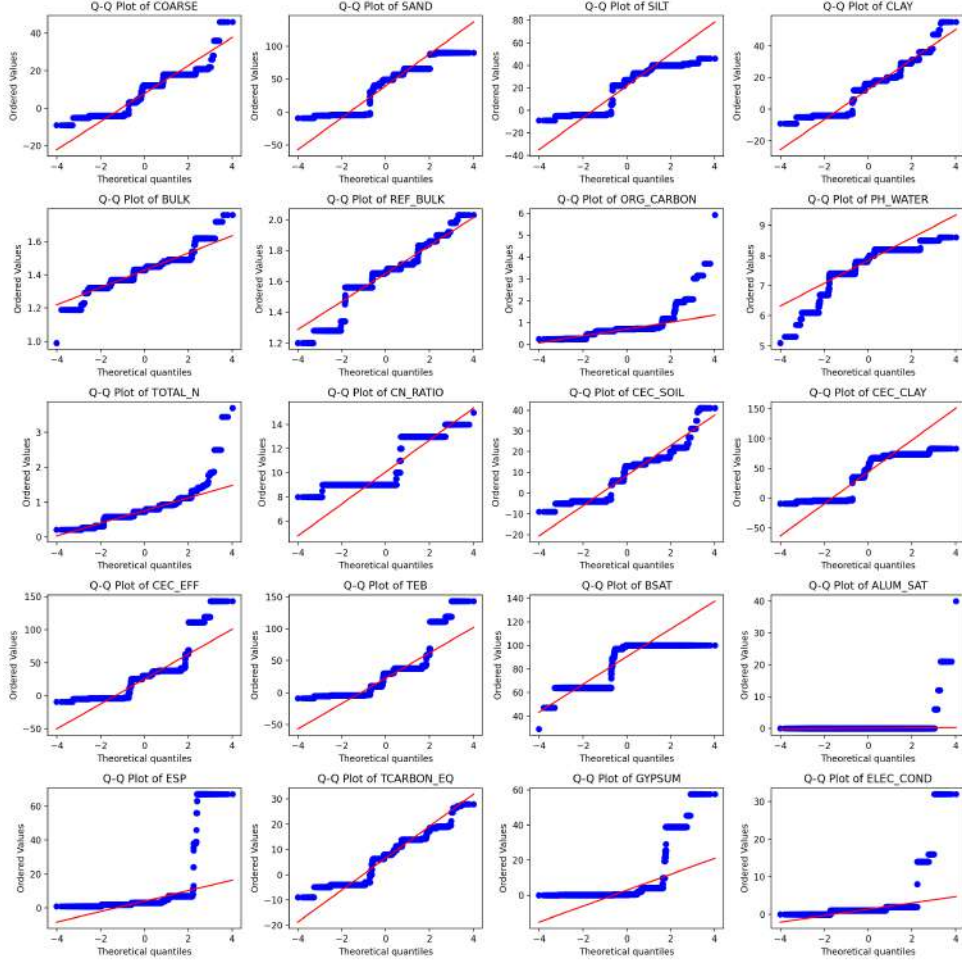


Figure A.12: Quantiles plots of numeric soil variables after imputation, highlighting deviations from normality.

A.7 Resampling Methods

Synthetic Minority Over-sampling Technique (SMOTE)

SMOTE (Synthetic Minority Over-sampling Technique) is an over-sampling approach that generates new synthetic samples for the minority class instead of simply duplicating existing observations. For each minority instance, synthetic points are created by interpolating between the sample and its k nearest minority neighbors in feature space.

This method increases the representation of the minority class while preserving the underlying data structure, thereby reducing overfitting risks associated with naive duplication. In this study, SMOTE was applied with $k = 5$ nearest neighbors, resulting in a perfectly balanced training set where fire and non-fire classes contain an equal number of samples.

SMOTE is particularly useful when the minority class is under-represented but sufficiently dense to allow meaningful interpolation.

Tomek Links Under-sampling

Tomek Links is an under-sampling technique designed to clean ambiguous samples near class boundaries. A Tomek link is defined as a pair of samples from opposite classes that are mutual nearest neighbors. Such pairs typically lie in overlapping regions and may correspond to noisy or borderline observations.

When Tomek Links are removed, the majority-class samples involved in these pairs are eliminated, resulting in a cleaner and more separable dataset. In contrast to SMOTE, Tomek Links does not balance the dataset but slightly reduces class overlap while preserving the original class ratio.

In this work, Tomek Links was used to evaluate whether boundary cleaning alone could improve class separability without altering the overall imbalance structure.

Combined SMOTE + Tomek Links

The SMOTE+Tomek strategy combines the strengths of both over-sampling and under-sampling. First, SMOTE is applied to generate synthetic minority samples and balance the dataset. Then, Tomek Links are identified and removed to clean overlapping regions created during the over-sampling process.

This hybrid approach aims to achieve both class balance and improved class separation. In the context of wildfire prediction, this method provides a more robust training set by increasing minority representation while simultaneously reducing noise near decision boundaries.

As shown in Figure 4.3, SMOTE+Tomek yields a balanced dataset similar to SMOTE, but with slightly fewer samples due to the removal of ambiguous points.

Comparison and Practical Considerations

The resampling strategies serve different purposes:

- **Original:** Preserves the true data distribution but strongly favors the majority class.
- **SMOTE:** Balances classes through synthetic generation, improving recall of fire events.
- **Tomek Links:** Cleans class boundaries without enforcing balance.
- **SMOTE+Tomek:** Balances the dataset while reducing class overlap, often yielding more stable classifiers.

A.8 K-Nearest Neighbors (KNN) Model Visualizations

From-Scratch KNN Model Evaluation

Confusion Matrices (Training, Validation, and Test Sets) The training confusion matrix shows perfect classification, with no false positives or false negatives, indicating complete memorization of the training data. In contrast, the validation and test confusion matrices reveal a more realistic performance. Both sets exhibit a non-negligible number of false positives, while false negatives remain relatively limited. This behavior reflects a model that prioritizes sensitivity to fire events at the expense of precision, a common outcome in imbalanced classification settings. The similarity between validation and test error patterns suggests stable generalization behavior.

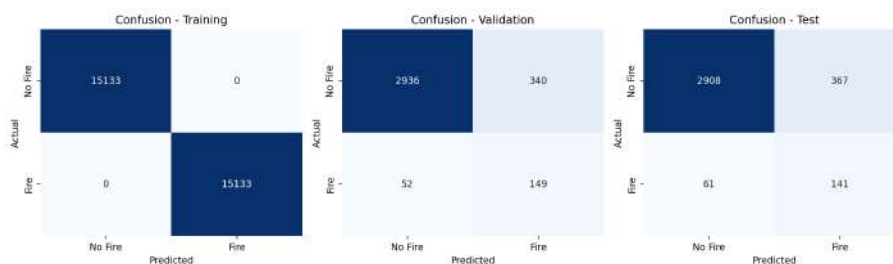


Figure A.13: Confusion matrices for the from-scratch KNN model.

ROC Curve Analysis The ROC curves further confirm these observations. The training ROC curve achieves an AUC of 1.00, reinforcing the presence of overfitting. However, the validation and test ROC curves achieve AUC values of approximately 0.885 and 0.864, respectively, indicating good discriminative ability on unseen data. Both curves remain well above the random classifier baseline, demonstrating that the from-scratch KNN model effectively ranks fire-prone and non-fire observations despite reduced threshold-dependent performance.

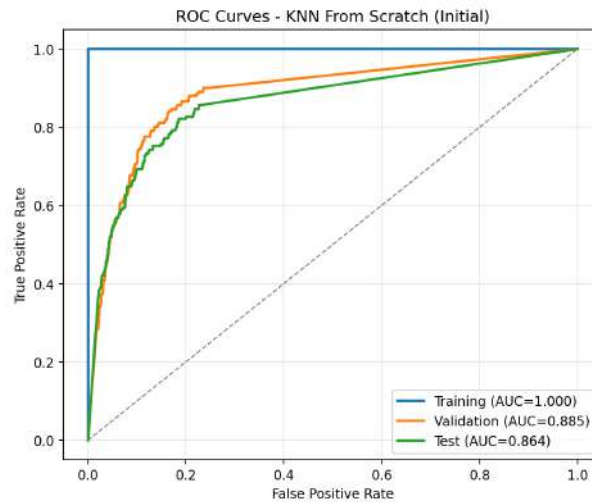


Figure A.14: ROC curves for the from-scratch KNN model.

KNN Model Performance Metrics

Training performance is perfect across all metrics, indicating clear overfitting. Validation and test accuracy remain high; however, precision and F1-score are substantially lower due to class imbalance. Recall is moderate, while specificity remains high, showing strong performance on the majority class. MCC values indicate limited but non-random predictive capability on unseen data.

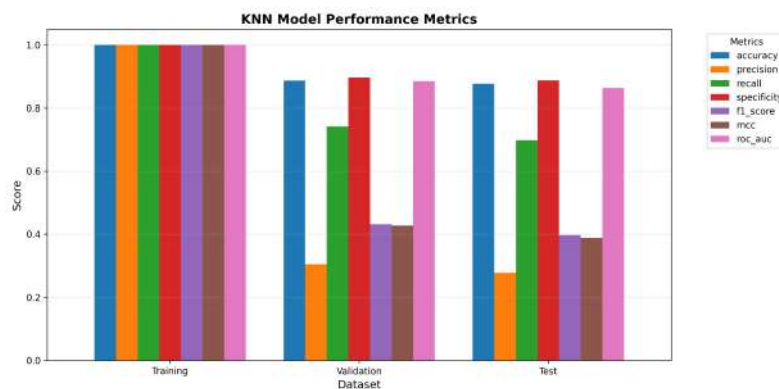


Figure A.15: Comparison of performance metrics for the predefined KNN model.

Confusion Matrices

The training confusion matrix shows perfect classification with no errors. Validation and test matrices exhibit a limited number of false negatives and a higher number of false positives, leading to reduced precision but acceptable recall. Error patterns are consistent across validation and test sets.

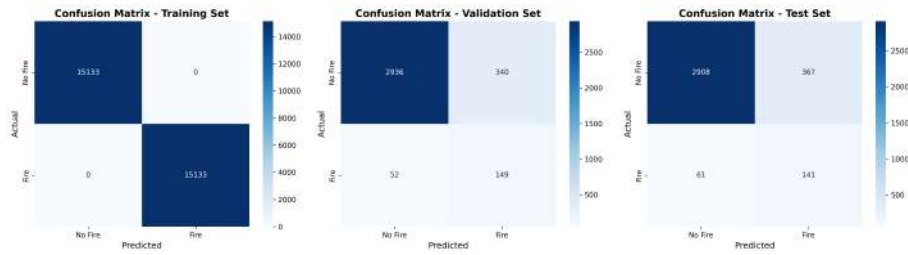


Figure A.16: Confusion matrices for the predefined KNN model.

ROC Curves

The training ROC curve achieves an AUC of 1.00, confirming overfitting. Validation and test ROC curves achieve AUC values of approximately 0.89 and 0.86, respectively, indicating good discriminative performance. Both curves remain well above the random classifier baseline.

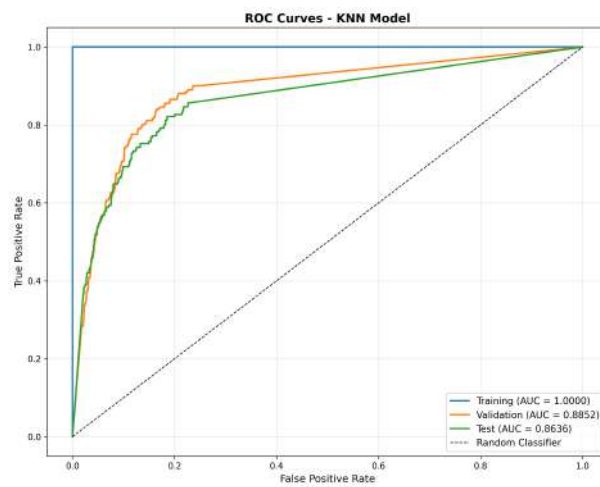


Figure A.17: ROC curves for the predefined KNN model.

Threshold Optimization Analysis

Increasing the classification threshold results in decreasing recall and increasing precision and specificity. The F1-score and MCC reach their maximum values near the selected threshold of 0.50. False negatives increase with higher thresholds, highlighting the cost of conservative decision boundaries.

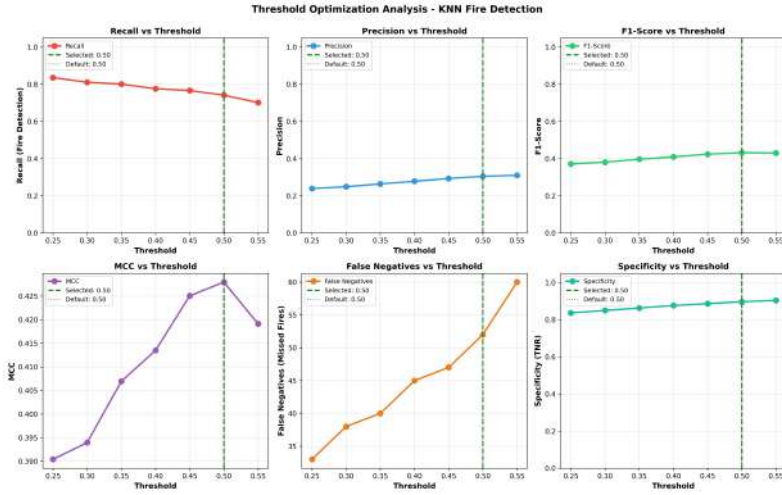


Figure A.18: Bayesian threshold optimization results for the predefined KNN model.

Predefined vs. Bayesian Optimized KNN Comparison

Bayesian optimization yields marginal improvements in accuracy, specificity, precision, and F1-score on validation and test sets. Recall decreases slightly, reflecting a trade-off favoring precision. ROC-AUC shows minor degradation, indicating optimization focused on threshold-dependent metrics rather than ranking performance.

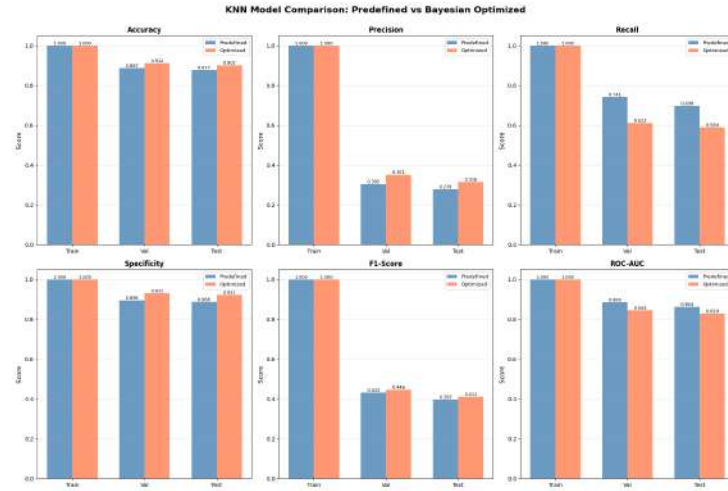


Figure A.19: Comparison of performance metrics between predefined and Bayesian optimized KNN models.

Bayesian Optimization of KNN

The Bayesian optimization process converges rapidly, with F1-score stabilizing early, indicating an efficient search. The distribution of F1-scores is tightly concentrated near the optimum (approximately 0.96), demonstrating stable model performance across evaluated configurations.

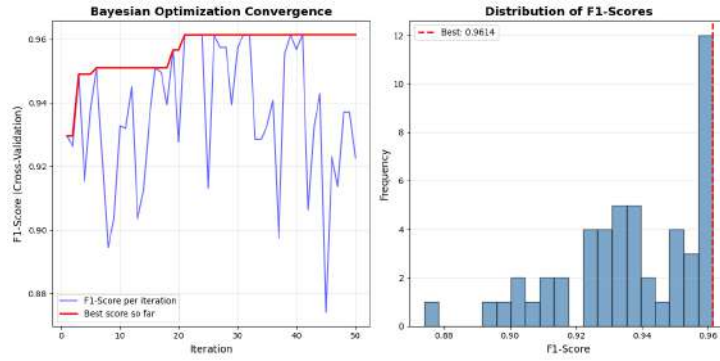


Figure A.20: Bayesian optimization process for the predefined KNN model.

Overall Interpretation

The KNN model exhibits strong ranking ability but is affected by overfitting and class imbalance. Bayesian optimization and threshold tuning improve precision-oriented metrics and stability at the expense of recall, making the final model suitable for applications prioritizing false-positive control.

A.9 Decision Tree

A.9.1 From scratch Implementation

As discussed in the chapter, the Tomek dataset achieved the best performance among the three resampling strategies when using the scratch Decision Tree implementation.

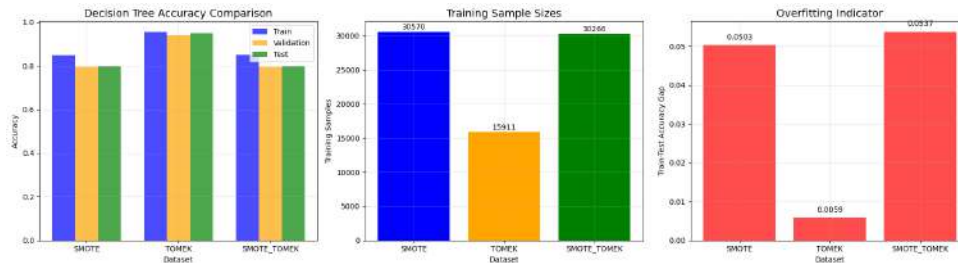


Figure A.21: Decision Tree (Scratch) Performance Comparison Across Resampling Strategies

A.10 Random forest

A.10.1 From-Scratch Implementation

Graphical Analysis

Metric Comparison Plot The figure below illustrates the comparison of the main evaluation metrics across the three resampling strategies. Tomek Links consistently achieves the highest overall performance, particularly in terms of accuracy and F1-score.

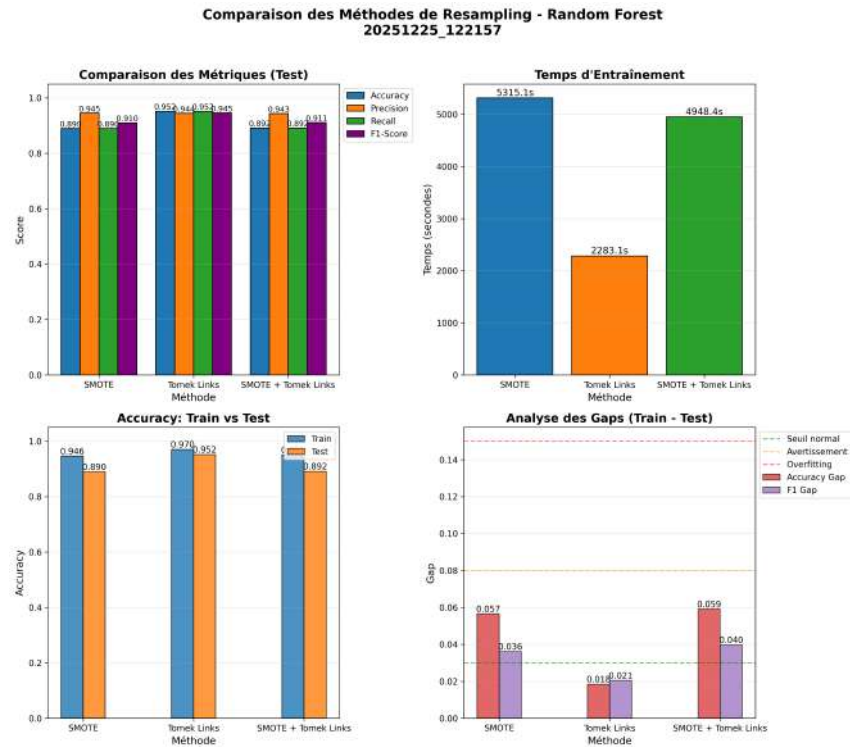


Figure A.22: Random Forest (From Scratch) – Comparison of Resampling Methods

Overfitting Analysis

The gap between training and test performance was analyzed to assess overfitting. Tomek Links exhibits the smallest generalization gap (**Accuracy gap = 0.0185**, **F1 gap = 0.0205**), indicating excellent generalization compared to SMOTE-based approaches, which show mild overfitting.

Radar Chart Analysis The radar chart below provides a holistic view of the trade-offs between precision, recall, F1-score, accuracy, and computational cost. Tomek Links demonstrates the best balance between predictive performance and training time (biggest and roundest polygon).

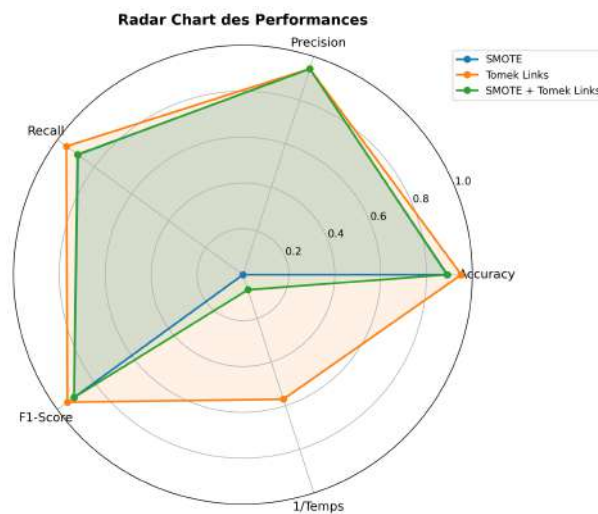


Figure A.23: Random Forest (From Scratch) – Radar Chart Comparison

A.10.2 Scikit-learn Implementation

Table A.2: Scikit-Learn Random Forest Class-wise Precision and Recall

Resampling	Class 0 Precision	Class 0 Recall	Class 1 Precision	Class 1 Recall
SMOTE	0.98	0.91	0.33	0.68
Tomek	0.96	0.98	0.57	0.36
SMOTE-Tomek	0.98	0.91	0.31	0.66

A.10.3 Bayesian Optimization for Random Forest

Bayesian Optimization was used to tune Random Forest hyperparameters on the SMOTE-Tomek dataset. The search space included `n_estimators` (50–200), `max_depth` (5–25), and `max_features` (0.1–0.9).

Table A.3: Selected Iterations from Bayesian Optimization for Random Forest

Iteration	CV Accuracy	n_estimators	max_depth	max_features
1	0.9691	106	24.01	0.686
3	0.9678	59	22.32	0.581
6	0.9695	109	24.21	0.636
10	0.9703	71	25.00	0.100
16	0.9710	175	25.00	0.100
19	0.9710	159	25.00	0.100

Bayesian Optimization completed in approximately 419 seconds. The optimal hyperparameters were `n_estimators=159`, `max_depth=25`, `max_features=0.1`, `random_state=42`, and `n_jobs=-1`.

A.10.4 Discussion

The Random Forest classifier achieved consistently high performance on the majority class (Class 0) across all resampling strategies, with SMOTE slightly improving minority class recall. Bayesian optimization further improved validation accuracy, achieving 0.9336 on the SMOTE-Tomek dataset. Training times were reasonable, and Random Forest proved to be robust for handling class imbalance in forest fire prediction tasks.

A.11 Visualisations of K-Means results

A.11.1 Bayesian Optimization and Optimized K-Means ($K = 8$)

Bayesian Optimization Convergence The optimization objective remains constant across iterations, indicating that the Bayesian optimization process did not effectively explore the hyperparameter space.

Parameter Space Exploration All tested values of K yield nearly identical scores. As a result, the selection of $K = 8$ is not statistically supported by the optimization results.

Optimized Clustering Analysis The first principal component explains 99.83% of the total variance, indicating that the data is effectively one-dimensional. The presence of vertically aligned clusters in the PCA projection suggests over-segmentation and confirms that $K = 8$ exceeds the intrinsic structure of the data.

Clustering Metrics Comparison Compared to the default configuration ($K = 3$), the Silhouette score decreases (from 0.71 to 0.60), indicating reduced cluster separation. Although the Davies–Bouldin index improves and the Calinski–Harabasz index increases, these improvements primarily reflect increased compactness and sensitivity to higher K rather than better structural clustering.

Inertia Analysis A significant reduction in inertia is observed for $K = 8$. However, since inertia decreases monotonically with the number of clusters, this result alone does not indicate improved clustering quality.

Summary While $K = 8$ produces more compact clusters according to internal metrics, it leads to over-segmentation and reduced interpretability. The optimized solution is therefore mathematically tighter but structurally less meaningful.

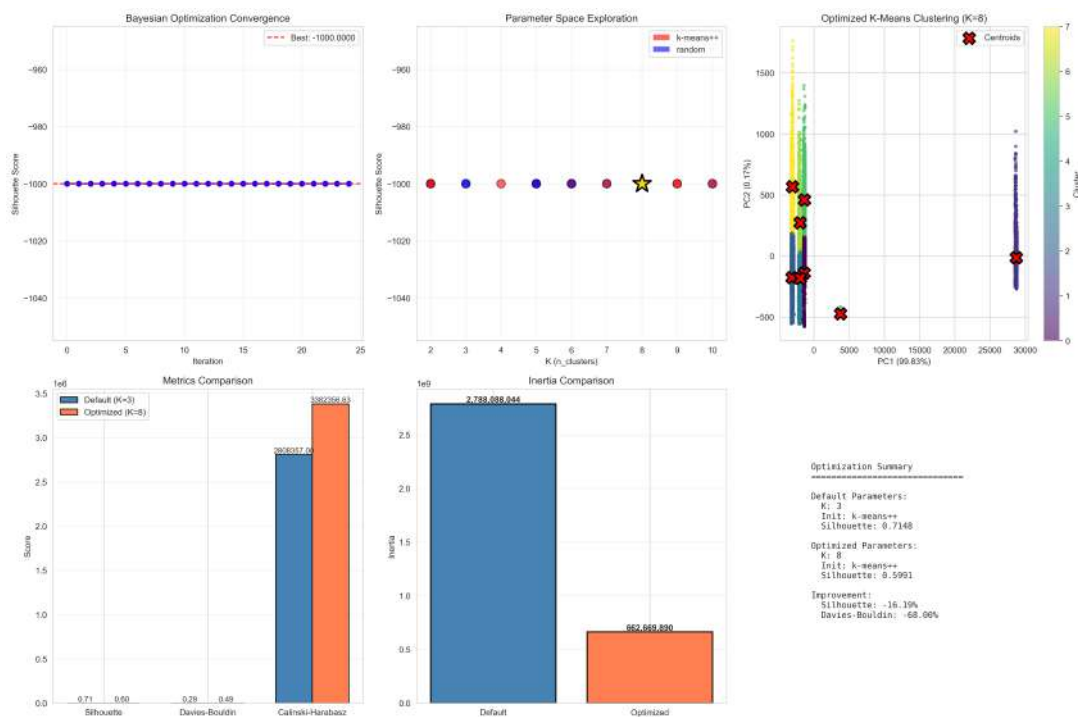


Figure A.24: Bayesian optimization applied to K-Means hyperparameters.

A.11.2 From-Scratch K-Means Clustering ($K = 5$)

Cluster Visualization (PCA Projection) The PCA projection reveals one dominant cluster containing approximately 64% of the samples, indicating a strong imbalance. Smaller clusters primarily capture outliers or edge cases rather than well-defined groups. Cluster centroids are mainly aligned along the first principal component, confirming the near one-dimensional structure of the data.

Cluster Size Distribution The cluster size distribution is highly imbalanced, suggesting that the clustering is driven by data density rather than underlying structure. This behavior indicates that the natural grouping of the data is closer to 2 or 3 clusters rather than 5.

Silhouette Analysis The average Silhouette score is approximately 0.60, reflecting acceptable but moderate clustering quality. The wide dispersion of silhouette values within clusters indicates internal inconsistency and partial overlap between clusters.

Performance Summary The algorithm achieves a reasonable runtime; however, the resulting clustering is more suitable for coarse segmentation than detailed profiling. The choice of $K = 5$ already exceeds the structural capacity of the data.

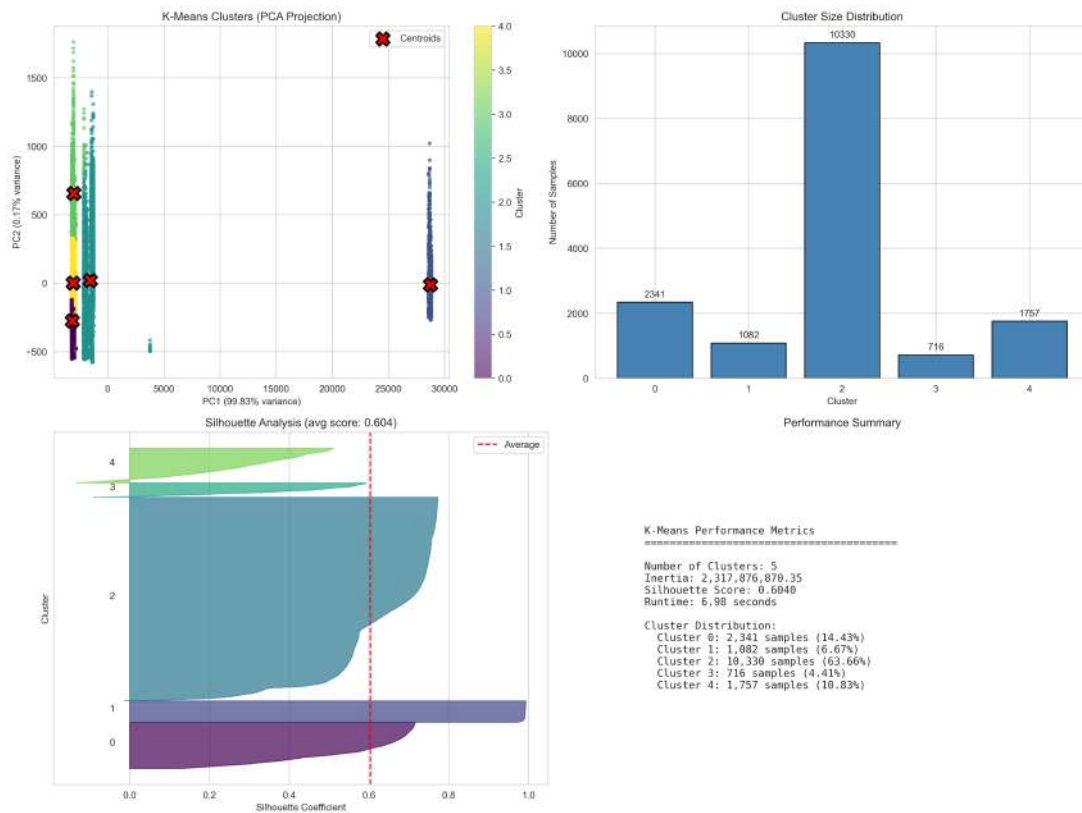


Figure A.25: Visualization of K-Means clustering implemented from scratch.

A.11.3 Scikit-Learn K-Means Clustering ($K = 3$)

Cluster Visualization (PCA Projection) The PCA projection shows clear separation along the first principal component. Clusters are compact and well-defined, in agreement with the intrinsic low-dimensional structure of the data.

Cluster Size Distribution Although the cluster sizes remain imbalanced, the distribution is structurally meaningful. The dominant cluster reflects the true data density rather than overfitting or artificial fragmentation.

Silhouette Analysis The average Silhouette score is high (approximately 0.715), with a narrow distribution of values, indicating strong internal cohesion and well-separated clusters. This configuration exhibits the best consistency between visual and quantitative evaluations.

Performance Summary The $K = 3$ configuration provides the best balance between cluster separation, stability, and interpretability. The reduced number of clusters results in improved generalization.

A.11.4 Overall Clustering Assessment

The scikit-learn K-Means solution with $K = 3$ is the most appropriate configuration for this dataset. Bayesian optimization did not yield meaningful improvements due to a flat objective

surface and low metric sensitivity. Increasing the number of clusters mainly improves mechanical metrics such as inertia and the Calinski–Harabasz index, while degrading structural coherence. Given the effectively one-dimensional nature of the data, higher values of K lead to unnecessary over-segmentation.

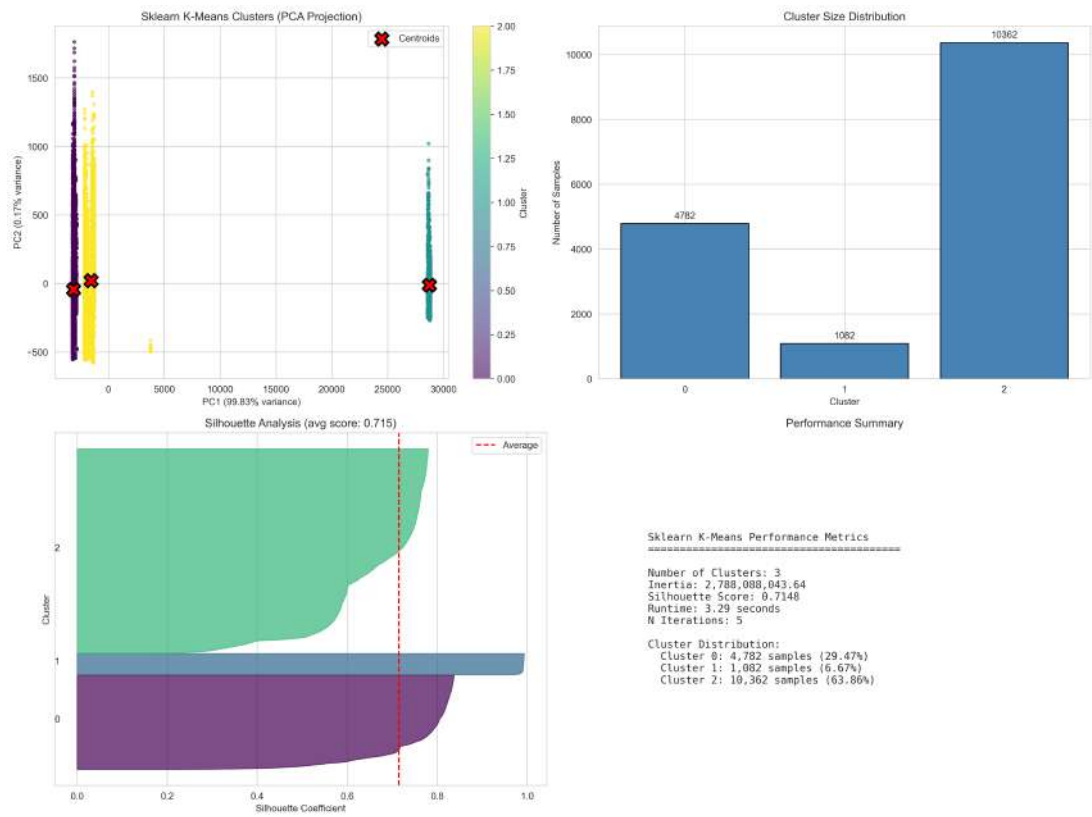


Figure A.26: Visualization of K-Means clustering using scikit-learn implementation.

A.12 DBSCAN Clustering Visualizations

A.12.1 DBSCAN with Alternative Parameterization

DBSCAN Clusters (PCA) A reduced number of clusters is obtained, with the majority of samples assigned to a single dominant cluster and only a minimal proportion labeled as noise.

Cluster Size Distribution One cluster overwhelmingly dominates the distribution, indicating a loss of density discrimination under this parameter configuration.

Fire Rate per Cluster Fire rates vary across clusters; however, the dominant cluster closely reflects the global fire rate due to its size.

Contingency Table Fire events are largely concentrated within the dominant cluster, limiting the discriminative capacity of the clustering.

Silhouette Analysis The average Silhouette score remains negative, confirming weak separation and internal inconsistency among clusters.

DBSCAN Summary This configuration reduces noise but collapses the clustering structure into a single dominant cluster, resulting in limited analytical value.

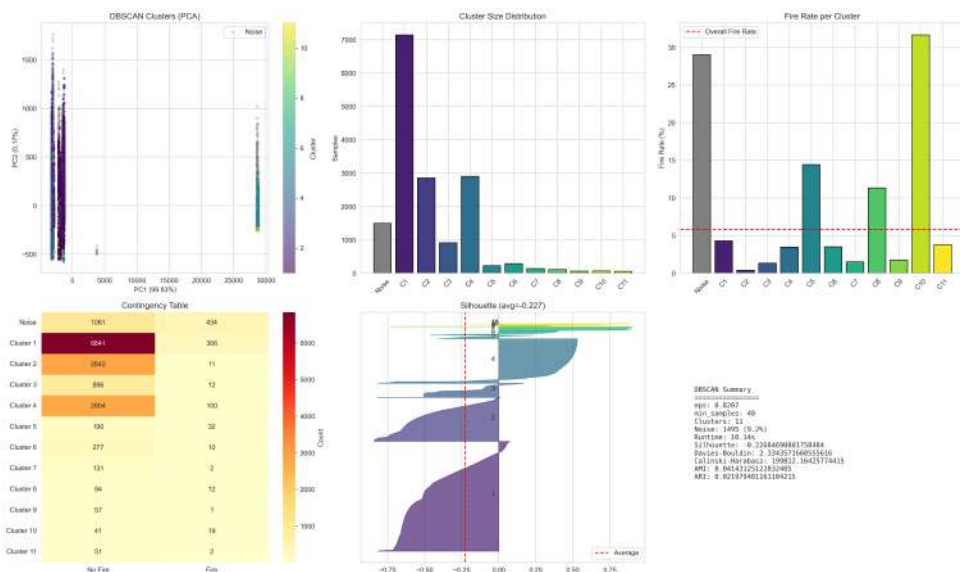


Figure A.27: Analysis of DBSCAN clustering implemented from scratch.

A.12.2 DBSCAN vs. Ground Truth (PCA Projection, Scikit-Learn) and (PCA Projection, From-Scratch)

DBSCAN Clusters (PCA) DBSCAN identifies a limited number of dense regions primarily along the first principal component, while labeling a large proportion of samples as noise. Cluster boundaries do not align clearly with the global data structure.

Ground Truth Labels (PCA) Ground truth fire labels are widely dispersed across the PCA space with no clear linear separability, indicating a complex and overlapping class structure.

DBSCAN Clusters (PCA) The from-scratch implementation yields a similar cluster geometry, characterized by a strong dominance of noise points and limited cluster cohesion.

Ground Truth Labels (PCA) Fire and non-fire samples remain interwoven across the feature space, confirming the absence of clear density-based separation.

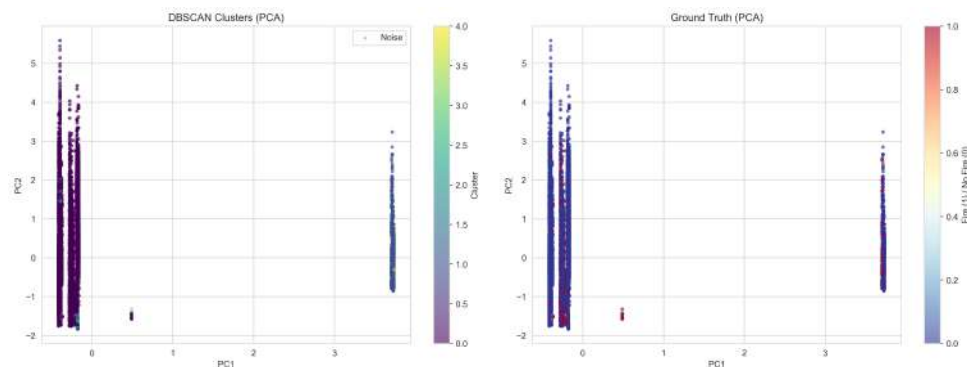


Figure A.28: DBSCAN clustering results after PCA, compared to ground truth.

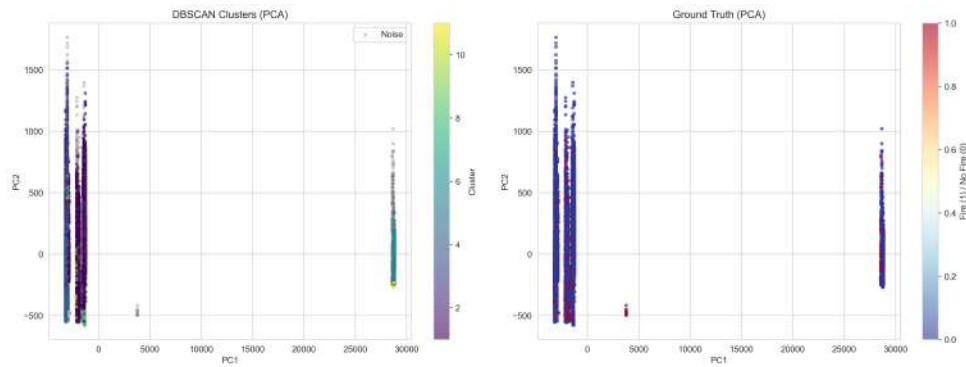


Figure A.29: DBSCAN clustering from scratch after PCA, compared to ground truth.

A.12.3 DBSCAN Detailed Analysis (Optimized Configuration)

DBSCAN Clusters (PCA Projection) The clustering reveals multiple small clusters along with a non-negligible proportion of noise points. Cluster separation is primarily driven by the first principal component.

Cluster Size Distribution Cluster sizes are highly imbalanced, with a small number of dominant clusters and several very small ones, indicating heterogeneous density regions within the data.

Fire Rate per Cluster Fire occurrence varies significantly across clusters. Some clusters exhibit fire rates well above the global average, highlighting heterogeneous risk distributions.

Contingency Table (Fire vs. No Fire) The majority of samples correspond to non-fire instances across all clusters. Fire events are unevenly distributed, with certain clusters capturing a disproportionately higher number of fire cases.

Silhouette Analysis The average Silhouette score is negative, indicating substantial overlap between clusters and weak overall separation.

DBSCAN Summary Although the runtime remains acceptable, clustering quality is limited, as evidenced by negative Silhouette values and low agreement metrics.

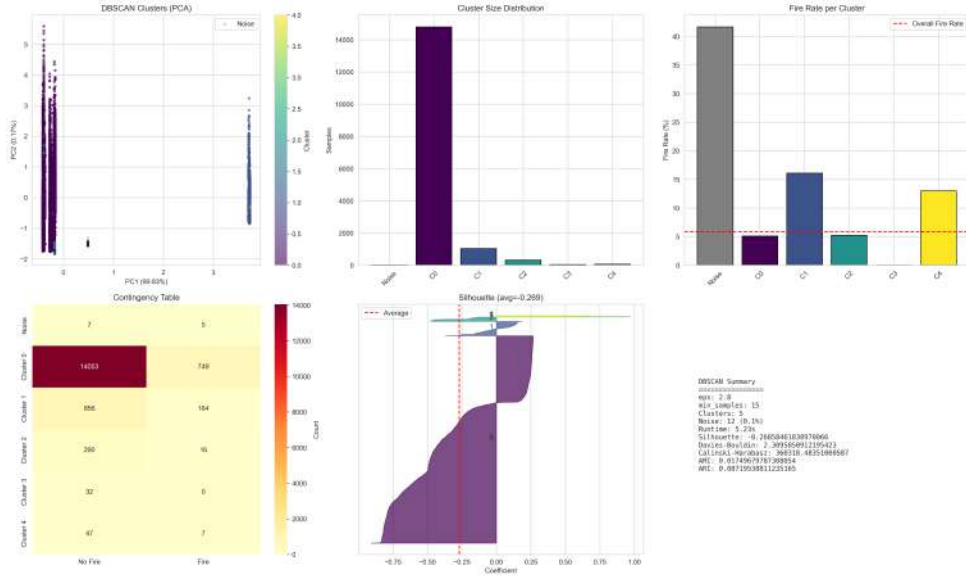


Figure A.30: Analysis of DBSCAN clustering using scikit-learn implementation.

A.12.4 Bayesian Optimization and Optimized DBSCAN

Bayesian Optimization Convergence The Silhouette score increases across iterations and stabilizes near its maximum value, indicating effective exploration and convergence of the Bayesian optimization process.

Parameter Space Exploration (ϵ , `min_samples`) Higher Silhouette scores are observed for specific combinations of moderate ϵ values and higher `min_samples`. The selected configuration corresponds to a local optimum in the parameter space.

Optimized DBSCAN Clustering (PCA Projection) Clusters are mainly separated along the first principal component, which explains most of the variance. Several small clusters and noise points are detected, reflecting the sensitivity of DBSCAN to local density variations.

Metrics Comparison (Default vs. Optimized) The optimized configuration improves the Silhouette score and reduces the Davies–Bouldin index. The Calinski–Harabasz index decreases, which is expected given DBSCAN’s non-globular clustering structure and the presence of noise.

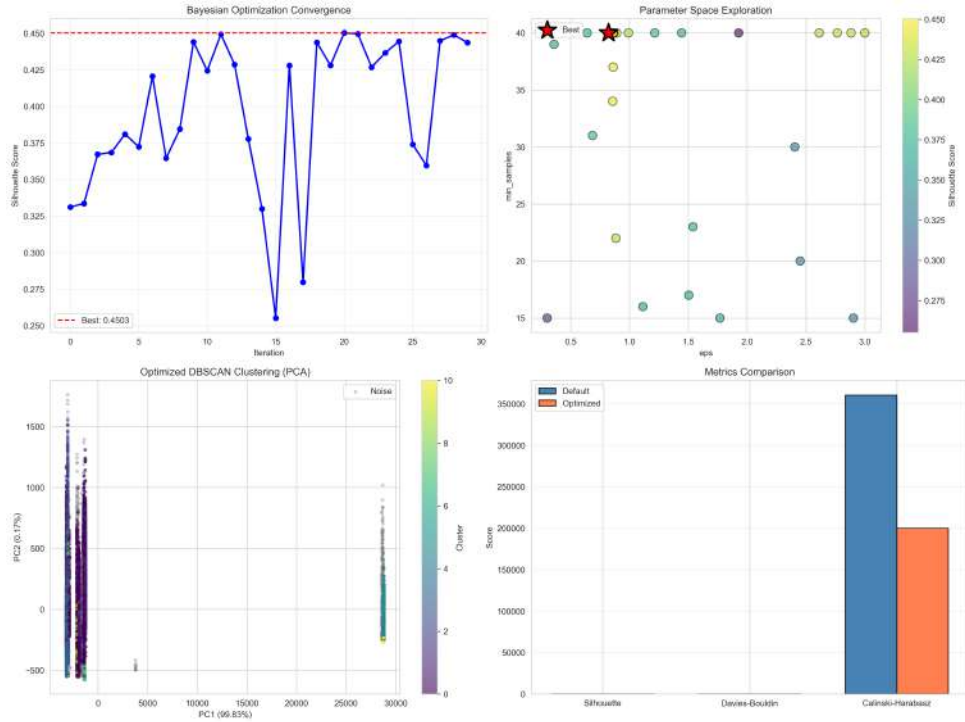


Figure A.31: Bayesian optimization applied to DBSCAN hyperparameters.

A.12.5 Overall DBSCAN Conclusion

Although Bayesian optimization improves certain internal metrics, DBSCAN fails to produce well-separated and stable clusters for this dataset. The effectively one-dimensional data structure and overlapping class distributions substantially limit the effectiveness of density-based clustering methods.

A.13 Clara K-Medoids Clustering

A.13.1 Scikit-learn Implementation

Optimal Cluster Number Selection

To determine the optimal number of clusters, CLARA was evaluated across multiple values of k (from 2 to 4) using various clustering metrics. The evaluation was performed on 3,000 randomly sampled points from each dataset.

Table A.4: Clustering Metrics for Different Values of k Across Datasets

Dataset	k	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
SMOTE	2	0.9711	0.0301	458,267.66
	3	0.6791	0.3226	574,914.17
	4	0.5353	0.7200	486,408.59
Tomek	2	0.9694	0.0311	265,918.53
	3	0.7163	0.2871	547,820.34
	4	0.5706	0.5876	508,135.20
SMOTE-Tomek	2	0.9711	0.0302	470,223.54
	3	0.6695	0.3355	532,586.24
	4	0.5430	0.6655	467,548.81

For all three datasets, $k = 2$ consistently achieved the highest Silhouette Score (>0.96) and lowest Davies-Bouldin Index (<0.032), indicating excellent cluster separation and compactness. Although $k = 3$ yielded higher Calinski-Harabasz scores, the substantial drop in Silhouette Score (to 0.67-0.72) suggests over-segmentation. Based on these results, $k = 2$ was selected as the optimal configuration for all datasets.

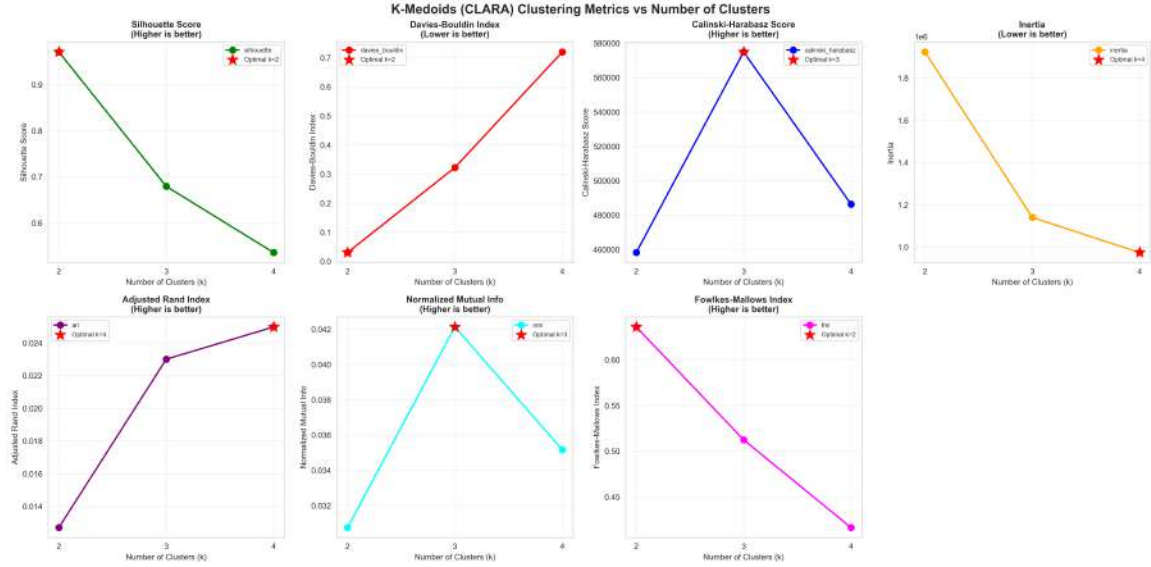


Figure A.32: Clustering Metrics vs. Number of Clusters for SMOTE Dataset. The optimal value of $k = 2$ is marked with red stars across all metrics.

Tomek dataset

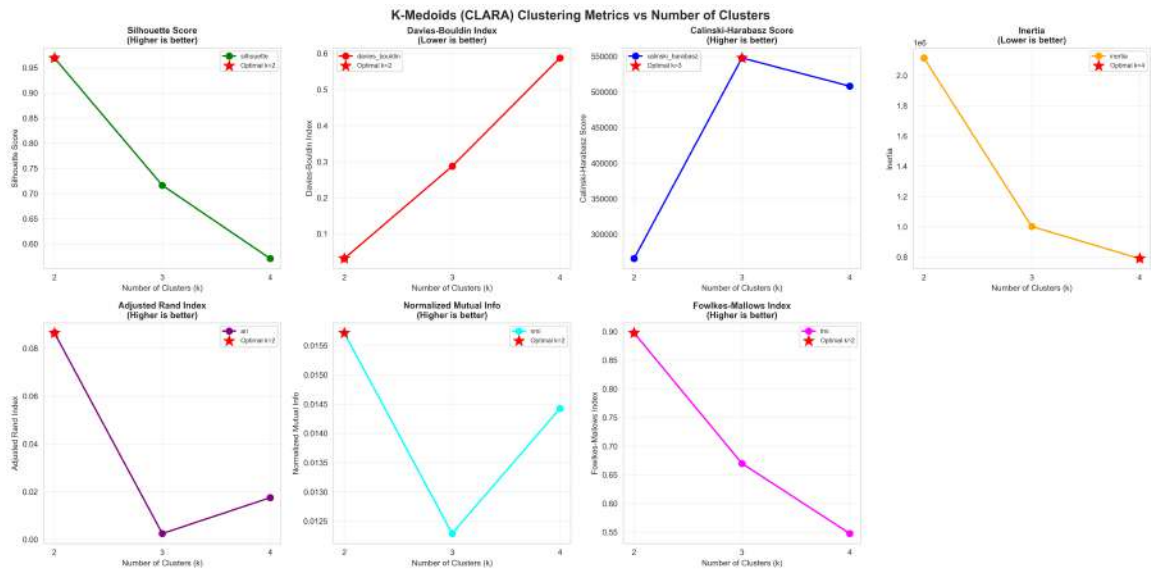


Figure A.33: Clustering Metrics vs. Number of Clusters for TOMEK Dataset. The optimal value of $k = 2$ is marked with red stars across all metrics.

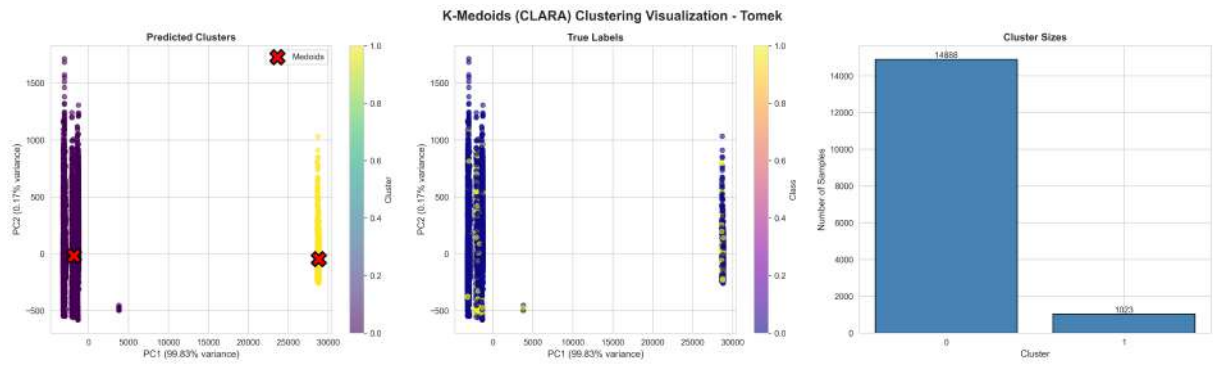


Figure A.34: CLARA Clustering Visualization for TOMEK Dataset ($k = 2$). Left: Predicted clusters with medoids marked as red crosses. Center: True class labels for comparison. Right: Cluster size distribution showing balanced partitioning.

Smote-Tomek dataset

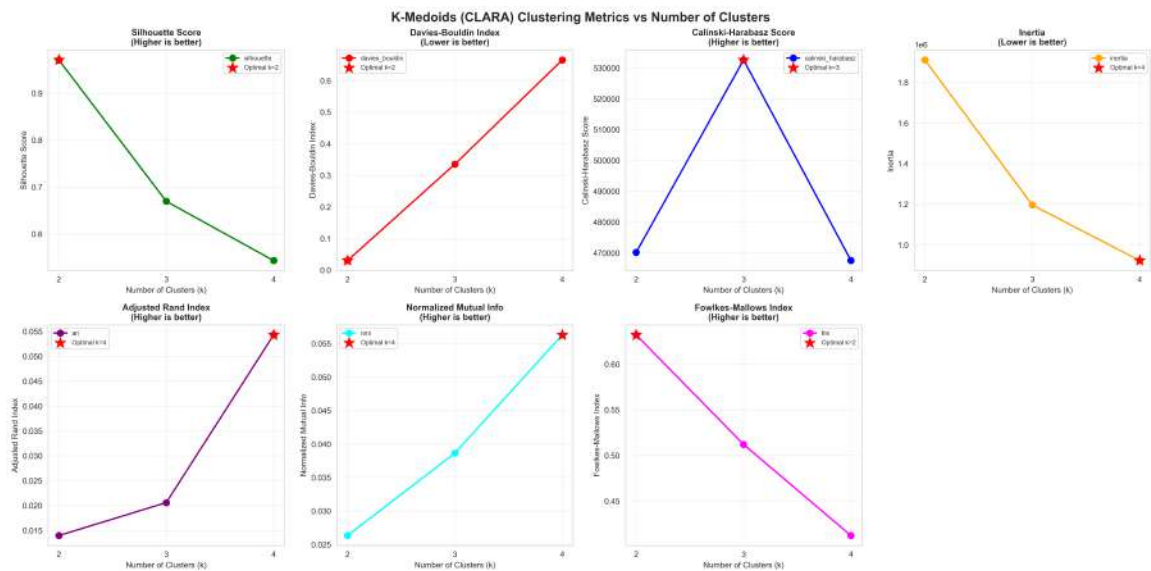


Figure A.35: Clustering Metrics vs. Number of Clusters for SMOTE-TOMEK Dataset. The optimal value of $k = 2$ is marked with red stars across all metrics.

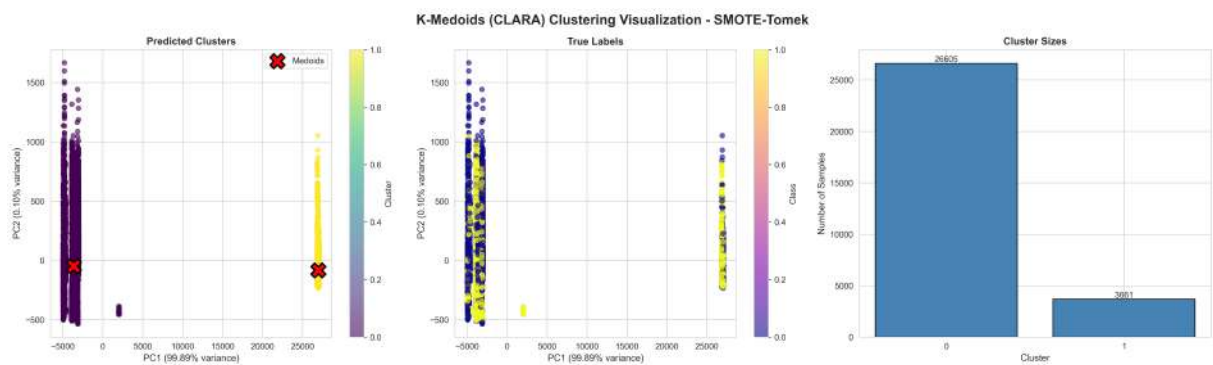


Figure A.36: CLARA Clustering Visualization for SMOTE-TOMEK Dataset ($k = 2$). Left: Predicted clusters with medoids marked as red crosses. Center: True class labels for comparison. Right: Cluster size distribution showing balanced partitioning.

Cross-Dataset Comparison

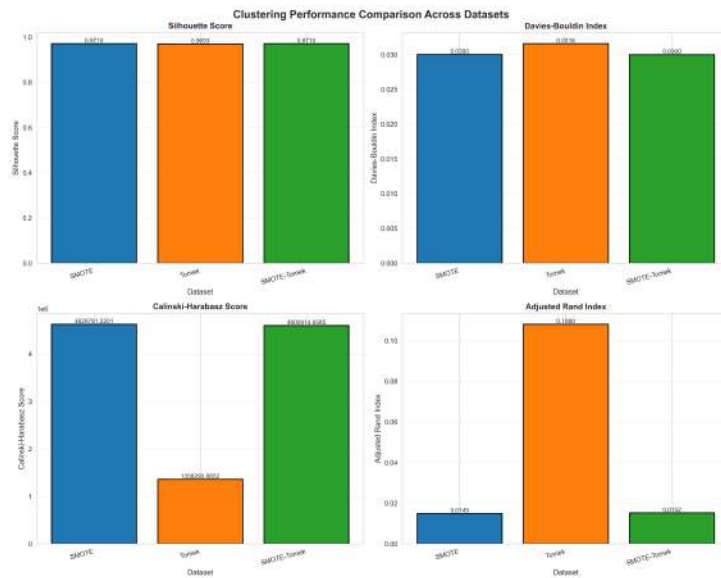


Figure A.37: Performance Comparison of CLARA Across Resampling Methods. All datasets achieved similar Silhouette Scores (>0.96), while Tomek showed superior alignment with true labels (highest ARI and Fowlkes-Mallows Score).

The comparison across resampling methods reveals consistent clustering quality, with all datasets achieving Silhouette Scores exceeding 0.96 and Davies-Bouldin indices below 0.032. The Tomek dataset's superior ARI (0.1080) and Fowlkes-Mallows Score (0.8967) suggest that removing noisy boundary samples enhances the results. The SMOTE and SMOTE-Tomek datasets, while producing equally compact clusters, show lower external validation metrics, indicating that synthetic sample generation introduces variability that creates natural groupings different from the original class structure.

A.13.2 Bayesian Optimization

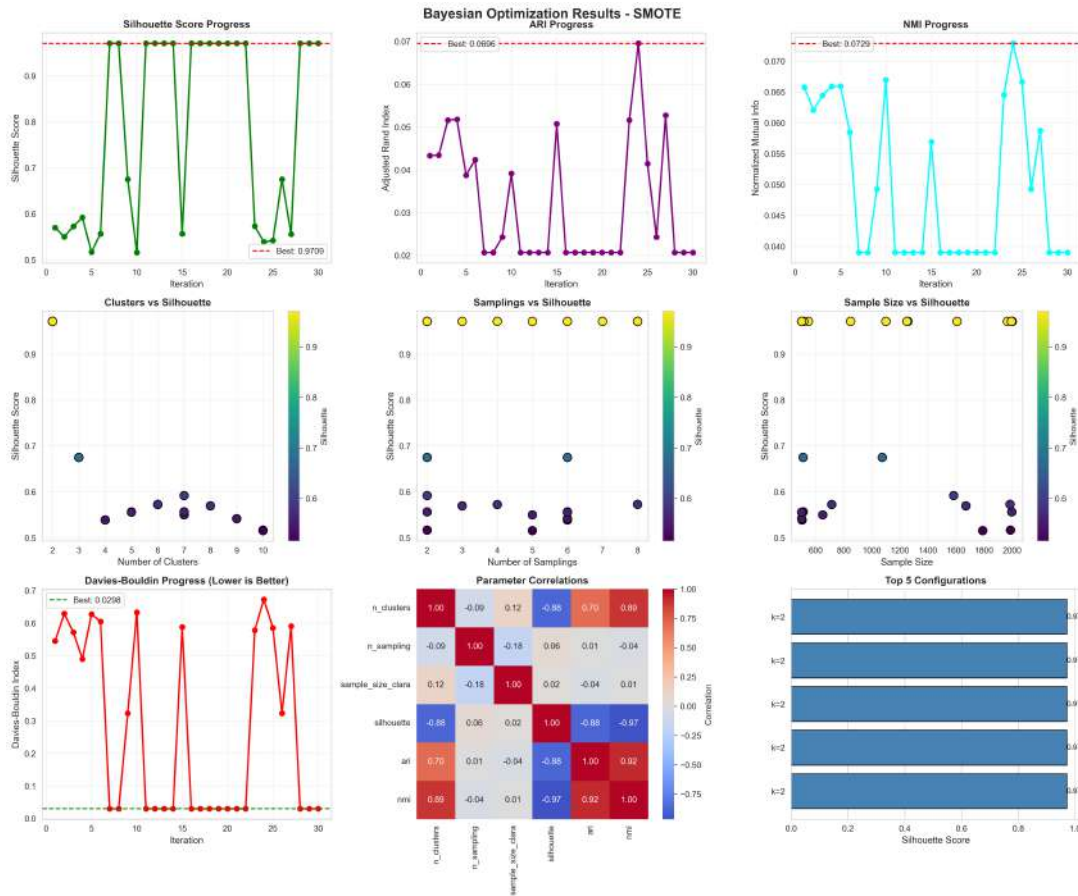


Figure A.38: Bayesian Optimization Progress for SMOTE Dataset. Top row: Evolution of clustering metrics across 30 iterations. Bottom row: Parameter impact on Silhouette Score, parameter correlation heatmap, and top 5 configurations. The algorithm rapidly converged to $k = 2$ after exploring higher values of k in early iterations.

The optimization trace reveals that the algorithm quickly identified the superior performance of $k = 2$ (Silhouette Score 0.97) compared to larger cluster counts ($k \geq 3$: Silhouette 0.54-0.67). After the initial exploration phase (iterations 1-6), Bayesian Optimization predominantly sampled $k = 2$ configurations, demonstrating efficient convergence. The parameter correlation heatmap shows weak correlations between hyperparameters, indicating that the number of clusters dominates performance, while sampling parameters have minimal impact within reasonable ranges.

Tomek dataset

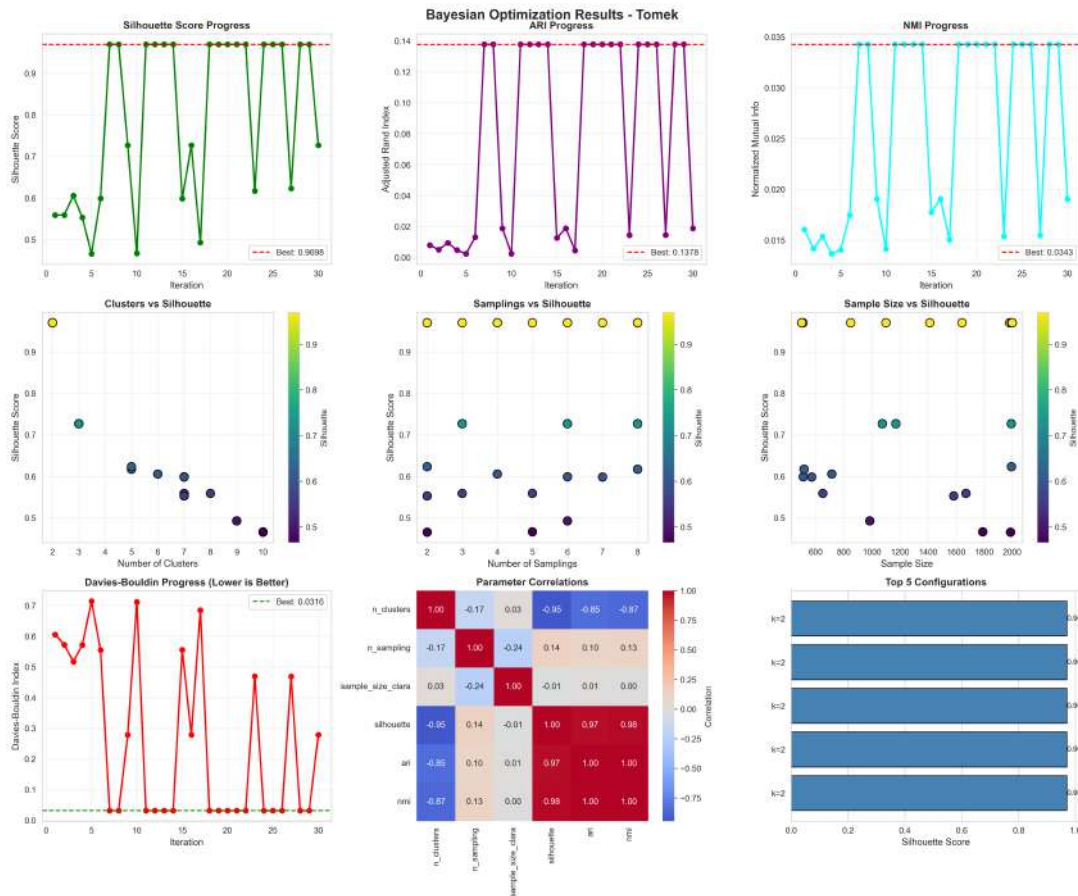


Figure A.39: Bayesian Optimization Progress for TOMEK Dataset. Top row: Evolution of clustering metrics across 30 iterations. Bottom row: Parameter impact on Silhouette Score, parameter correlation heatmap, and top 5 configurations. The algorithm rapidly converged to $k = 2$ after exploring higher values of k in early iterations.

Smote-Tomek dataset

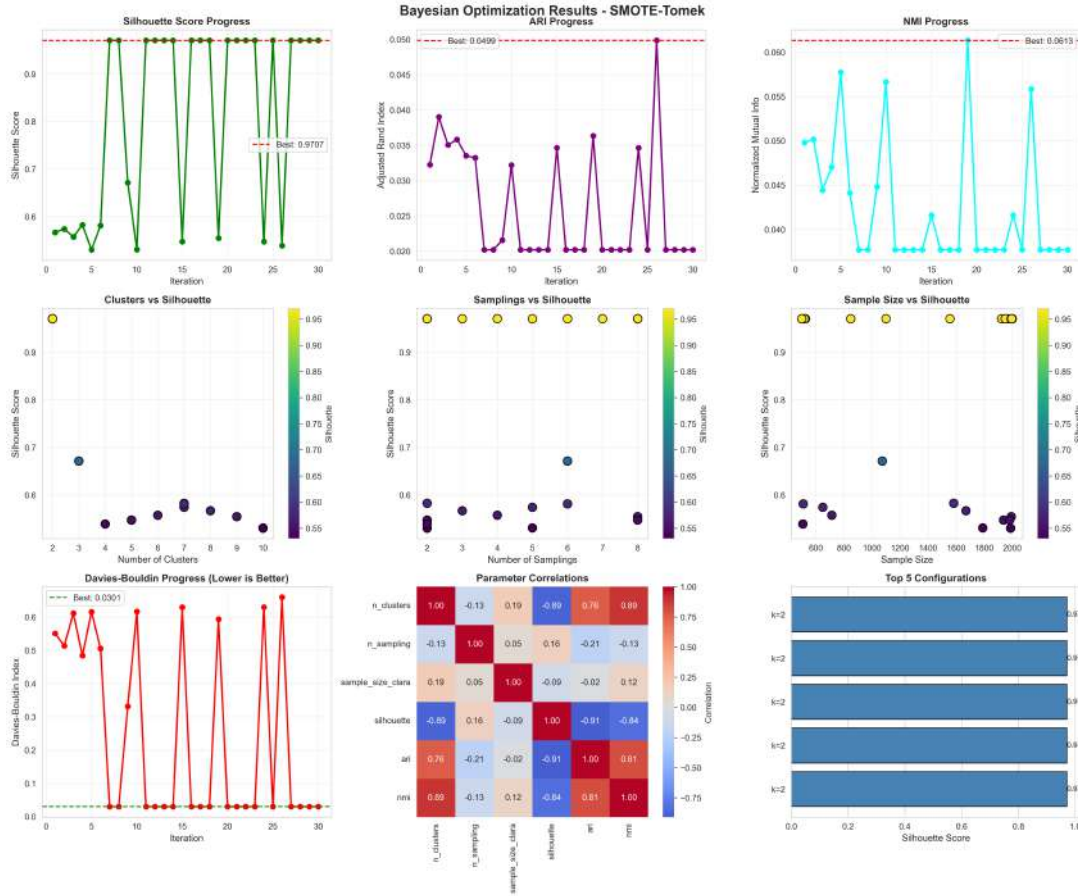


Figure A.40: Bayesian Optimization Progress for SMOTE-TOMEK Dataset. Top row: Evolution of clustering metrics across 30 iterations. Bottom row: Parameter impact on Silhouette Score, parameter correlation heatmap, and top 5 configurations. The algorithm rapidly converged to $k = 2$ after exploring higher values of k in early iterations.

Comparison of all datasets

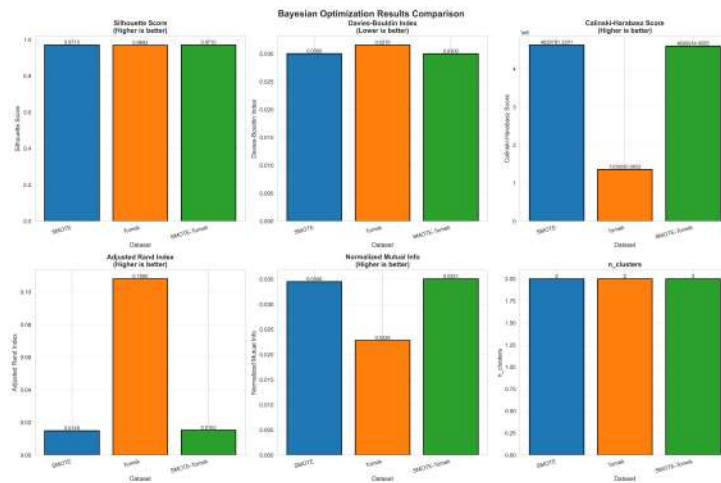


Figure A.41: Performance Comparison of Bayesian Optimization Across Resampling Methods.

A.13.3 From-Scratch Implementation of CLARANS

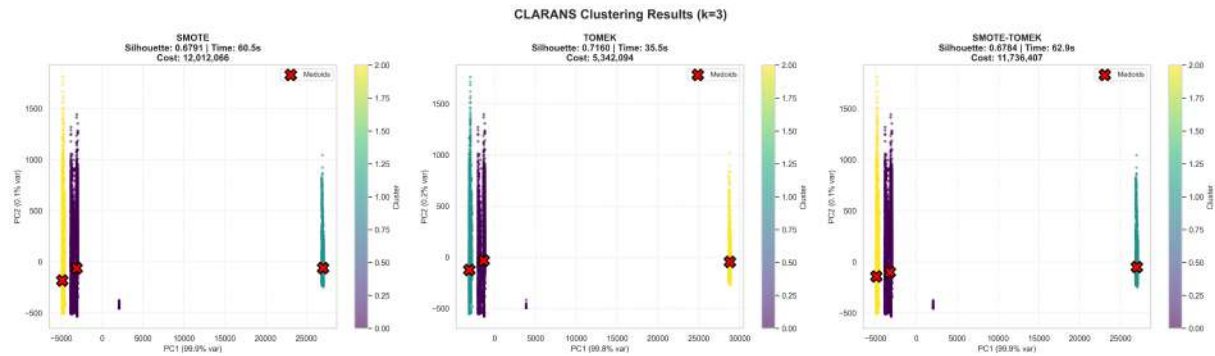


Figure A.42: 2D PCA Projection of CLARANS Clustering Results ($k = 3$) on All Three Datasets. Each subplot shows predicted clusters (color-coded), medoids (red crosses), and cluster boundaries. The Tomek dataset (center) exhibits the most distinct cluster separation, as reflected in its superior Silhouette Score.

The PCA visualizations reveal the internal structure discovered by CLARANS. In the Tomek dataset, three clusters are clearly separated with minimal overlap in the 2D projection, consistent with its high Silhouette Score (0.7160). The SMOTE and SMOTE-Tomek datasets show more cluster overlap, particularly in the central region where synthetic samples were generated to balance class distributions. The medoids (marked as red crosses) are strategically positioned at the centers of dense regions, demonstrating CLARANS' effectiveness in identifying representative cluster prototypes. The first two principal components capture approximately 40-50% of the total variance across all datasets, indicating that while the 2D projections provide useful insights, the full 64-dimensional feature space contains additional structure that contributes to the clustering results.

Notably, the randomized local search strategy required exploring 353-973 neighbor configurations per local run before convergence, with the Tomek dataset requiring the most extensive search (973 neighbors in the final local run). This higher exploration effort likely contributed to finding the superior medoid configuration reflected in its better clustering metrics. The algorithm's ability to escape local minima through multiple independent local searches ($numlocal = 3$) proved effective, with 2 out of 3 runs yielding cost improvements across the datasets.

Overall, the from-scratch CLARANS implementation successfully validated the algorithm's ability to discover meaningful cluster structures in high-dimensional fire detection data. The results demonstrate that data preprocessing strategy (SMOTE vs. Tomek vs. combined) significantly impacts clustering quality, with boundary-cleaning approaches (Tomek) producing more separable clusters than synthetic oversampling methods.

- S. et al. Archibald. Soil fertility and african fire patterns. *Journal of Biogeography*, 2013.
- N. Batjes. *SOTER Soil Database Framework*, 2016.
- D. et al. Bowman. Fire ecology and soil texture effects. *Science*, 2009.
- N. Brady and R. Weil. *The Nature and Properties of Soils*. Pearson, 2017.
- G. Certini. Soil physical properties and fire susceptibility. *Catena*, 2005a.
- G. Certini. Fire effects on soil ph. *Oecologia*, 2005b.
- W. et al. De Groot. Soil organic carbon and wildland fires. *Fire Ecology*, 2009.
- FAO and IIASA. *Harmonized World Soil Database v2: Technical Report*, 2021.
- C. Fernandez and J. Vega. C:n ratio and litter flammability. *Forest Ecology and Management*, 2018.
- L. Giglio, J. Randerson, and G. van der Werf. Analysis of global fire patterns. *J. Geophys. Res.*, 2013.
- J. et al. Herrero. Gypsiferous soils in arid lands. *Catena*, 2009.
- Hans Jenny. *Factors of Soil Formation*. McGraw-Hill, 1941.
- USDA NRCS. *Soil Texture Triangle*, 2017.
- Ohio State University Extension. Base saturation and cation exchange capacity in soils (anr-81), 2008. URL <https://ohioline.osu.edu/factsheet/anr-81>.
- Stephen M. Omohundro. Five balltree construction algorithms. *International Computer Science Institute*, 1989.
- J. Pausas and R. Bradstock. Nutrient cycling and fire regimes. *Global Ecology and Biogeography*, 2012.
- I. et al. Prieto-Fernandez. Soil moisture and fire susceptibility. *Environmental Modelling*, 2019.
- M. et al. Qadir. Soil sodicity and vegetation cover. *Land Degradation and Development*, 2014.
- J. et al. Reynolds. Soil compaction and fire recovery. *Geoderma*, 2018.

- J. et al. Ruiz-Sinoga. Salinity, vegetation, and fire in drylands. *Journal of Arid Environments*, 2012.
- S. et al. Saatchi. Soil texture and fire risk modeling. *Remote Sensing of Environment*, 2012.
- scikit-learn developers. Nearest neighbors — ball tree. <https://scikit-learn.org/stable/modules/neighbors.html>, 2023. Accessed 2025.
- R. W. Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.
- USDA Natural Resources Conservation Service. Soil bulk density, moisture, and aeration. Technical report, USDA NRCS, 2022. URL <https://www.nrcs.usda.gov/sites/default/files/2022-10/Soil%20Bulk%20Density%20Moisture%20Aeration.pdf>.
- J. P. Wilson and J. C. Gallant. Machine learning for soil science: A review. *Geoderma*, 401: 115206, 2021.